Integrated Model for Aircraft Routing and Maintenance Planning

Thesis Manuscript

August 18, 2026

Abstract

This thesis studies the integrated aircraft routing and maintenance planning problem within a compact mixed-integer programming framework. We begin from the compact tail-assignment model and show that the basic feasibility conditions are insufficient for physical realism because they do not fully exclude simultaneous departures and overlapping aircraft trajectories. We then formalize and correct the cumulative flight-hour maintenance constraint, which is the central modeling issue in the original formulation. In addition, the thesis extends the model to a full four-level maintenance hierarchy using A/B/C/D checks, while maintaining the structure of a compact and operationally meaningful assignment model. The resulting formulation is evaluated against constructive and search-based heuristics and benchmarked on generated instance families designed to capture varying density and maintenance pressure. The study shows that the corrected maintenance logic yields valid schedules, tighter formulations, and a more reliable comparison between exact and heuristic strategies.

Acknowledgements

The author acknowledges the value of the underlying modeling and computational work that motivated this thesis. The research is built on the existing compact tail-assignment formulation and its maintenance extension, with emphasis on mathematical correctness, operational feasibility, and reproducible computational evidence.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Introduction

The airline planning process is traditionally decomposed into a sequence of sub-problems: timetable design, fleet assignment, crew rostering, tail assignment, and recovery planning [**?** **?**]. The *tail assignment problem* (TAP)—assigning a specific aircraft (identified by its tail number) to each flight leg—is the penultimate planning step and is tightly coupled with aircraft maintenance scheduling. Poor tail assignment decisions directly inflate operational costs through unnecessary ferry (repositioning) flights, avoidable leasing decisions, suboptimal maintenance routing, and service disruptions.

?] proposed a compact 0–1 linear programming formulation for the TAP, featuring only $O(n_f n_p)$ binary variables (where n_f is the number of flights and n_p the fleet size). This is a significant reduction over the classic multi-commodity flow model of **?**], which requires $O(n_f^2 n_p)$ variables. Computational experiments reported in that paper show that instances with up to 1 500 flight legs and 40 aircraft can be solved to certified optimality within one hour on a standard workstation.

However, most practical studies still rely on cyclic or short-horizon settings (daily/weekly schedules) and often separate routing from maintenance decisions. For modern airline operations, this is restrictive: schedules are frequently adjusted for seasonality and event-driven demand, while aircraft must satisfy multi-level maintenance obligations across the same horizon. Treating routing and maintenance independently shifts feasibility issues into the operational window, where corrective actions are expensive and disruptive.

This thesis addresses that gap by revisiting and extending the compact TAP framework toward an integrated, season-level routing-and-maintenance planning approach. The objective is to produce physically executable aircraft rotations that satisfy maintenance requirements and improve planning certainty before schedule execution.

Thesis contributions

This thesis makes four contributions that verify, correct, and extend the state of the art:

1. **Basic feasibility correction (C2–C4 / constraints (3)–(6)).** We show that the strict-time balance definition used in the compact basic model admits a simultaneous-departure corner case: one aircraft can be assigned to two flights departing at the same airport and same time without violating the balance inequalities. We formalize this counter-example and enforce physical executability by adding explicit pairwise non-overlap constraints.

2. **Constraint (13) correction.** We show that the cumulative-flight- time maintenance constraint (Constraint 13 in **?**]) contains a logical flaw: the big-M linearisation term $M(2 - y_{jd} - y_{jd'})$ leaves the constraint vacuously satisfied whenever at least one of the endpoint days d, d' hosts no check, so an accumulation window that is not bounded by checks at both ends is left unconstrained and may exceed the flight-hour limit. We provide a corrected formulation using two split endpoint constraints, prove its validity, and show that the corrected formulation yields strictly tighter LP relaxations on high-density instances.

3. **Full $\{A, B, C, D\}$ maintenance hierarchy.** The model of **?**] handles type-A checks (cumulative flight hours). We extend the formulation to the full four-level hierarchy mandated by aviation regulators (FAA/EASA): type-A and B checks are flight-hour triggered; type-C and D checks are calendar-day triggered. A heavier check subsumes all lighter checks (hierarchical counter resets). We further add a constraint accounting for pre-horizon accumulated maintenance hours/days per aircraft and integrate these requirements into long-horizon schedule planning.

4. **Heuristic benchmark.** We implement and evaluate two constructive heuristics: (a) a greedy/insertion heuristic operating directly on flight lists, and (b) a Dijkstra-based heuristic on the space-time network representation (adapted from **?**]). Both heuristics respect the full maintenance hierarchy. We report optimality gaps against the MILP on the same instance sets and show the Dijkstra heuristic consistently outperforms an ACO alternative.

Thesis organisation

Chapter 2 reviews related work. Chapter 3 restates the basic TAP model and proves a C2–C4 corner-case correction. Chapter 4 presents the maintenance-extended model, the Constraint (13) correction, and the induced interpretation updates for C8–C14 under the corrected basic model. Chapter 5 describes the full $\{A, B, C, D\}$ extension.

Chapter 6 analyses the heuristic algorithms. Chapter 7 reports all computational results. Chapter 8 concludes.

Chapter 2

Literature Review

2.1 Literature Review

The problems studied in the domain of thesis can be divided into the following classes:

- Fleet assignment - selection of an aircraft type for each flight;
- Aircraft routing - construction of maintenance-feasible sequences for a homogeneous fleet;
- Tail assignment - allocation of individual aircraft, with their own locations and maintenance states, to those sequences; and
- Maintenance planning - scheduling checks and the tasks performed during them.

This distinction matters because models from these streams cannot be compared solely by instance size or solution time. They differ in decision scope, maintenance semantics, planning horizon, and treatment of uncertainty [**?** **?** **?**].

In the following sections, we review the literature on these four problem classes, as well as on integrated airline planning, robustness and operational uncertainty, and heuristic solution methods. The review concludes with a discussion of the research gap addressed by this thesis.

2.1.1 Aircraft routing and tail-assignment formulations

Early network formulations represent aircraft movement as flow through airport and time events. The multi-commodity model of **?**] tracks each aircraft separately, while the time-space network of **?**] provides a natural representation of flight, ground, and overnight arcs. Maintenance opportunities can then be embedded in feasible rotations [**?** **?**]. These models make physical continuity explicit, but aircraft-specific connection variables grow rapidly with the number of feasible flight pairs.

Route-based formulations instead associate variables with complete feasible flight strings. The flight-string model of **?**] exploits this structure through column generation. Such formulations can express detailed route feasibility in the pricing problem, but their implementation is substantially more involved than a directly solved compact MILP. Decomposition methods offer a related trade-off: they separate coupled decisions to obtain tractable subproblems, as illustrated by Benders approaches to aircraft routing and crew scheduling [**?** **?**].

Compact formulations avoid explicit route enumeration. The lifted daily maintenance-routing model of **?**], for example, strengthens a compact nonlinear formulation through reformulation and linearisation. **?**] pursue a different compact representation in which binary variables assign flights directly to individual aircraft and aggregate time-balance constraints impose route continuity without connection-arc variables. Its small variable count and direct solvability make it the closest baseline for this thesis. Compactness, however, transfers more logical work to aggregate constraints; consequently, corner cases and conditional maintenance bounds require particularly careful verification.

The daily model of **?**] is an important operational counterpoint. It assigns aircraft to one-day lines of flight, enforces current maintenance needs, and uses look-ahead constraints for the following two days. Exact and heuristic procedures can modify lines of flight to reduce maintenance misalignment under day-to-day perturbations. That work addresses operational flexibility around a supplied route plan, whereas the present thesis audits and extends a deterministic compact assignment formulation over its full planning horizon. The two approaches are therefore complementary rather than direct competitors.

2.1.2 Mathematical programming formulations

Three main modelling paradigms underlie TAP research:

Set partitioning / column generation. The non-compact formulation enumerates *strings* (feasible flight sequences per aircraft) as 0–1 variables. Because the number of strings is exponential, column generation is required [**?** **?** **?** **?**]. Combining column generation with branch-and-bound (*branch-and-price*) gives exact solutions but at significant algorithmic overhead. Robust extensions appear in **?**] and **?**].

Multi-commodity flow. **?**] introduced the compact multi-commodity flow formulation with $O(n_f^2 n_p)$ binary variables. This remains the standard benchmark for compact models and is the primary comparison point in **?**].

Time-space networks. **?**] proposed the time-space network for the *fleet assignment problem*. Nodes represent airport–time events; arcs encode ground waits, flights, and

overnight stays. Extensions to aircraft routing appear in **?**]. Preprocessing by node aggregation and island isolation reduces model size [**?** **?**]. Daily and weekly maintenance routing variants are studied in **?**] and **?**]; the latter also integrates fleet assignment decisions, but only approximate solutions are obtained via variable fixing.

Nonlinear / lifted formulations. **?**] present a compact formulation with nonlinear constraints that are linearised via the Reformulation–Linearisation Technique (RLT). The largest instances solved involve 344 flights, an order of magnitude below the instances handled by the model of **?**].

2.1.3 MILP alternatives to the Khaled compact model

Beyond the compact binary assignment model of **?**], several mixed-integer alternatives are relevant for exact TAP modelling and large-scale implementation:

- **Arc-flow MILP (time-indexed or event-indexed):** decision variables are attached to feasible flight-connection arcs instead of flight–aircraft assignment pairs, often yielding stronger network-structure constraints [**?** **?** **?**].
- **Route-based master MILP (set partitioning):** aircraft duties are represented as columns in a master MILP, typically solved with branch-and-price; this is exact but algorithmically heavier [**?** **?** **?**].
- **Decomposed MILP frameworks:** Benders- or Dantzig–Wolfe-style decompositions separate assignment/routing from maintenance-resource feasibility, improving scalability on larger fleets [**?** **?** **?**].
- **Two-stage or rolling-horizon MILP:** first assign flights, then repair/optimize maintenance and turnaround feasibility in a second MILP, trading global optimality for tractable runtime [**?** **?** **?**].
- **Robust/scenario MILP variants:** delay and disruption uncertainty is modelled through multiple scenarios with non-anticipativity-style linking constraints, producing schedules that are less brittle operationally [**?** **?**].

In this taxonomy, the Khaled model is best viewed as a compact, directly solvable baseline that is simple to implement and benchmark, while the alternatives above trade model size, decomposition complexity, and robustness for stronger scalability or richer operational realism.

2.1.4 Integrated airline planning

Sequential airline planning can propagate poor or infeasible decisions from fleet assignment to routing and then to crew planning. **?**] integrate fleet assignment, aircraft routing, and crew pairing in a single framework and solve it using Benders decomposition with acceleration strategies. Their results demonstrate the value of preserving cross-stage dependencies, but also show why decomposition is needed once fleet, route, and crew decisions are combined.

At a narrower integration level, **?**] jointly model maintenance routing and crew scheduling, including aircraft replacement, deadheading, night-flight allocation, and flight-hour maintenance thresholds. They compare Lagrangian relaxation and particle swarm optimisation, with the relaxation producing solutions closer to the small-instance optimum. These integrated models establish that routing decisions interact with downstream resources. They do not, however, provide a like-for-like test of the compact tail-assignment constraints studied here, because their variables, objectives, and resource constraints cover a broader planning problem.

2.1.5 Robustness and operational uncertainty

Deterministic cost-optimal routes can be fragile when delays propagate through tight aircraft connections. Recoverable-robust tail assignment explicitly balances planned performance with the ability to repair a disrupted schedule [**?**]. A simulation-optimisation line of work instead uses stochastic evaluation to place connection buffers and retime flights. The initial particle-swarm and Monte Carlo framework of **?**] was developed into the weekly hybrid model of **?**]. The latter combines a linearised maintenance-routing model with simulation-based retiming and reports substantial improvements in on-time performance and propagated delay on airline data.

These studies reveal a limitation of the present thesis: the corrected model establishes deterministic physical and maintenance feasibility, but it does not by itself guarantee resilience to stochastic delays. Robust retiming, recourse, and scenario-based evaluation are therefore complementary extensions, not features that can be inferred from a feasible deterministic schedule.

2.1.6 Maintenance planning at different decision levels

Aircraft maintenance enters the literature at three levels that should not be conflated. First, routing models ensure that an aircraft reaches an eligible station before a usage limit is exceeded. Second, long-term check scheduling selects dates and capacities for maintenance packages. Third, task allocation assigns individual inspection and repair tasks to those checks. The present thesis operates mainly at the first level, while using

a stylised hierarchy to represent interactions among check categories.

At the long-term level, **?**] use dynamic programming to coordinate A- and C-check schedules for a heterogeneous fleet while accounting for aircraft status and maintenance capacity. Their airline case study reports fewer checks and higher aircraft availability than current practice. At the task level, **?**] formulate repeated maintenance-task allocation as a time-constrained variable-sized bin-packing problem. Their worst-fit-decreasing heuristic obtains small gaps to an exact method while responding quickly enough for repeated replanning. **?**] then integrate check scheduling, task allocation, and shift planning in a decision support system and demonstrate improvements on airline cases.

This evidence also requires more precise terminology than a simple statement that four checks are universally mandated. Letter checks are practical packages whose content and intervals depend on the approved maintenance programme, aircraft type, utilisation, and operator. The supplied maintenance studies note that B-check tasks are often absorbed into successive A-checks and that some operators combine D-check work with heavy C-checks [**?** **?** **?**]. Accordingly, the A/B/C/D structure in Chapter 5 is a modelling abstraction for nested usage limits and counter resets. It is not a claim that every operator implements four separate maintenance events.

For this thesis, the maintenance-planning literature has two further implications. Check feasibility depends on both utilisation counters, such as flight hours and cycles, and calendar limits; and an aircraft's state before the planning horizon cannot be discarded. Conversely, a routing-level model that places a check does not solve hangar capacity, task packaging, workforce, or shift planning. Those resource layers remain outside the present formulation.

2.1.7 Heuristic solution methods and cross-domain transfer

Constructive and local-search methods are operationally useful when exact solution times are incompatible with replanning. **?**] develops tail-assignment heuristics in this setting. The original work targets a train application rather than flights, but the principles are transferable. The algorithm: represent movement and waiting in an acyclic space-time network, select locomotives in a cost-based order, repeatedly solve a least-cost path problem, update arc costs after assignment, and terminate a path when a maintenance limit becomes due. The algorithmic principles are applicable to aircraft because time also gives the routing network a natural topological order.

This transfer must preserve aircraft-specific feasibility. Airport continuity, turnaround time, aircraft eligibility, and all maintenance counters have to be encoded in nodes, arcs, or state labels before a shortest-path solution is a valid tail route. Dijkstra's shortest-path method provides the basic path engine [**?**]; ant colony optimisation

provides a population-based alternative for exploring assignment sequences [**?** **?**]. In this thesis these methods are benchmarked as constructive alternatives to the compact MILP, not treated as evidence of optimality.

2.1.8 Comparison scope and fairness

For an empirical comparison to be methodologically fair, competing models should be evaluated under a common instance family, objective definition, maintenance semantics, and solver-budget protocol. In TAP, this condition is often not met across compact MILP, branch-and-price, Benders-decomposed, and robust/scenario formulations because these approaches typically optimize different decision scopes or uncertainty assumptions. Accordingly, this thesis uses **?**] as the direct compact baseline for instance-matched comparisons, and treats other MILP families as complementary reference paradigms rather than strict one-to-one benchmarks.

2.1.9 Research gap and position of the thesis

The reviewed literature offers sophisticated solutions for route generation, integrated airline planning, robust retiming, long-term check scheduling, and maintenance-task allocation. Less attention is paid to auditing the logical correctness of a compact published formulation before extending or benchmarking it. This thesis addresses that narrower but foundational gap using **?**] as the direct baseline.

The contribution is positioned in four parts. First, the aggregate ground-flow conditions are tested against simultaneous and overlapping flight patterns, and explicit non-overlap conditions are introduced where needed. Second, the conditional cumulative-hour logic of Constraint (13) is analysed and replaced by two endpoint-anchored inequalities. Third, the maintenance state is extended to multiple nested check categories, including pre-horizon usage, while clearly remaining at routing level rather than modelling maintenance tasks or workforce. Fourth, exact and heuristic methods are evaluated on common generated instances under the same objective and feasibility checks.

This scope supports a controlled claim: the thesis improves the correctness and maintenance coverage of one compact TAP formulation and evaluates practical solution alternatives for that formulation. It does not claim to supersede route-based, decomposition, robust, or maintenance-resource models, whose broader decision scopes answer different operational questions.

Chapter 3

The Ground Routing Model

This chapter develops the ground-routing component of the thesis. The model is based on the compact tail-assignment formulation of **?**], but maintenance decisions are deliberately excluded until Chapter 4. This separation isolates the physical conditions that make a flight assignment executable and ensures that the maintenance extension is built on valid aircraft trajectories.

The term *compact* refers to the variable space. Route-based models use variables for complete rotations, and arc-flow models use variables for feasible flight connections. The formulation below instead uses one binary variable for each flight–aircraft pair. It therefore has $O(|\mathcal{F}||\mathcal{P}|)$ binary variables and can be submitted directly to a MILP solver. Airport continuity and timing are recovered through cumulative balance inequalities. This chapter derives those inequalities, identifies their equal-time departure limitation, and presents the corrected ground model used throughout the thesis.

3.1 Scope and modelling assumptions

The model is defined over a finite planning horizon using absolute times from the start of that horizon. A flight on day two consequently has larger time values than flights on day one. The following assumptions delimit the ground model:

1. every published flight leg must be covered exactly once;
2. each aircraft has one known initial airport;
3. an aircraft moves between airports only through an assigned flight, so any permitted ferry movement must be included explicitly;
4. departure and arrival times are fixed, and consecutive flights require at least the minimum ground turn time;
5. forbidden flight–aircraft pairs are removed or fixed to zero; and

6. maintenance and hangar decisions are deferred to Chapters 4 and 5.

The model is non-cyclic: an aircraft need not return to its initial airport by the end of the horizon. Its terminal position may be passed to the next planning period. Terminal-balance constraints can impose cyclicity, but this is an additional planning policy and may make a finite-horizon instance infeasible.

3.2 Sets, parameters, and cumulative flight sets

Let $\mathcal{F} = \{1, \dots, n_f\}$ be the set of flight legs, $\mathcal{P} = \{1, \dots, n_p\}$ the set of individual aircraft, and $\mathcal{A} = \{1, \dots, n_k\}$ the set of airports. For flight $i \in \mathcal{F}$, let $a_i^{\uparrow}$ and $a_i^{\downarrow}$ be its departure and arrival airports, and let $t_i^{\uparrow}$ and $t_i^{\downarrow}$ be its departure and arrival times. The arrow notation follows the flight event: $\uparrow$ denotes a departure and $\downarrow$ an arrival. Parameter a_j^0 is the initial airport of aircraft j, $\Delta t > 0$ is the minimum turn time, and c_{ij} is the cost of assigning flight i to aircraft j.

For airport $k \in \mathcal{A}$ and threshold θ, define

$$\mathcal{F}_k^{\downarrow}(\theta) = \{i \in \mathcal{F} : a_i^{\downarrow} = k,\ t_i^{\downarrow} \le \theta - \Delta t\}, \tag{3.1}$$

$$\mathcal{F}_k^{\uparrow}(\theta) = \{i \in \mathcal{F} : a_i^{\uparrow} = k,\ t_i^{\uparrow} < \theta\}. \tag{3.2}$$

The arrival set contains flights that have reached k early enough to complete the turn before θ. The departure set contains flights that have left k strictly before θ. The strict inequality is central to the formulation and to the counterexample in Section 3.5.

The cumulative inventory of aircraft j at airport k is

$$B_{jk}(\theta) = \sum_{i \in \mathcal{F}_k^{\downarrow}(\theta)} x_{ij} - \sum_{i \in \mathcal{F}_k^{\uparrow}(\theta)} x_{ij}. \tag{3.3}$$

At a non-initial airport, positive inventory can arise only from an earlier arrival. At a_j^0, one additional unit represents the initial aircraft.

3.3 Compact mixed-integer formulation

The binary assignment variable is

$$x_{ij} = \begin{cases} 1, & \text{if flight } i \text{ is operated by aircraft } j, \\ 0, & \text{otherwise,} \end{cases} \qquad i \in \mathcal{F},\ j \in \mathcal{P}. \tag{3.4}$$

Table 3.1: Notation for the compact ground-routing model.

Symbol	Meaning
$\mathcal{F}$, $\mathcal{P}$, $\mathcal{A}$	Flights, individual aircraft, and airports
$a_i^{\uparrow}$, $a_i^{\downarrow}$	Departure and arrival airports
$t_i^{\uparrow}$, $t_i^{\downarrow}$	Departure and arrival times
a_j^0	Initial airport of aircraft j
Δt	Minimum ground turn time
c_{ij}	Cost of assigning flight i to aircraft j
$\mathcal{F}_k^{\downarrow}(\theta)$	Turn-feasible arrivals at k by θ
$\mathcal{F}_k^{\uparrow}(\theta)$	Departures from k strictly before θ
$B_{jk}(\theta)$	Cumulative ground inventory

The objective minimises total assignment cost:

$$\min \quad \sum_{i\in\mathcal{F}} \sum_{j\in\mathcal{P}} c_{ij} x_{ij}. \tag{3.5}$$

The coefficients may represent operating cost or assignment preference. A forbidden pair must be removed or fixed to zero because a large cost alone does not make an assignment infeasible.

3.3.1 C1: flight coverage

Each flight is assigned to exactly one aircraft:

$$\sum_{j\in\mathcal{P}} x_{ij} = 1, \qquad \forall i \in \mathcal{F}. \tag{3.6}$$

This prevents both uncovered flights and duplicate coverage, but does not alone make the flights assigned to an aircraft a connected route.

3.3.2 C2: continuity at non-initial airports

For flight i departing from $k \neq a_j^0$, aircraft j must have reached k early enough and must not have consumed that arrival in an earlier departure:

$$\sum_{i'\in\mathcal{F}_k^{\downarrow}(t_i^{\uparrow})} x_{i'j} - \sum_{i'\in\mathcal{F}_k^{\uparrow}(t_i^{\uparrow})} x_{i'j} \geq x_{ij}, \qquad \begin{smallmatrix}\forall j\in\mathcal{P},\, k\in\mathcal{A}\setminus\{a_j^0\},\\ \forall i\in\mathcal{F}: a_i^{\uparrow}=k.\end{smallmatrix} \tag{3.7}$$

If $x_{ij} = 1$, this is equivalent to $B_{jk}(t_i^{\uparrow}) \geq 1$. Because only arrivals with $t_{i'}^{\downarrow} + \Delta t \leq t_i^{\uparrow}$ are counted, C2 also embeds the minimum turn-time requirement.

3.3.3 C3: continuity at the initial airport

At $k = a_j^0$, aircraft j is available once at the start of the horizon, so the right-hand side is shifted by one:

$$\sum_{i' \in \mathcal{F}_k^{\downarrow}(t_i^{\uparrow})} x_{i'j} - \sum_{i' \in \mathcal{F}_k^{\uparrow}(t_i^{\uparrow})} x_{i'j} \geq x_{ij} - 1, \quad \begin{smallmatrix} \forall j \in \mathcal{P},\, k = a_j^0, \\ \forall i \in \mathcal{F} : a_i^{\uparrow} = k. \end{smallmatrix} \tag{3.8}$$

The first departure can use the initial aircraft. Every later departure from the initial airport must be supported by a return arrival after accounting for earlier departures.

Finally,

$$x_{ij} \in \{0, 1\}, \qquad \forall i \in \mathcal{F},\ j \in \mathcal{P}. \tag{3.9}$$

3.4 Interpretation and validity of the balances

Constraints (3.7)–(3.8) replace explicit connection variables with airport inventories. A turn-feasible arrival adds one unit to (3.3); an earlier departure subtracts one unit. For distinct departure events, the inequalities implement conservation: an aircraft cannot depart a non-initial airport before arriving, and it cannot reuse an arrival after that inventory unit has been consumed.

The model does not select a predecessor flight explicitly. If several feasible arrivals precede a departure, the cumulative inequality certifies that one aircraft unit is present without naming the connection. This aggregation gives the model its small variable count, but makes correctness depend on event order.

Proposition 3.1 (Validity under distinct departure events)**.** *If no two flights depart from the same airport at the same time, every integral solution of* (3.6)–(3.9) *satisfies coverage, initial-position feasibility, airport continuity, and minimum turn time.*

Proof. Coverage follows from (3.6). Fix aircraft j and process its assigned departures in increasing time order. At a non-initial airport, (3.7) requires at least one more turn-feasible assigned arrival than earlier assigned departures. The current departure is therefore supported by an unused arrival at the same airport. At the initial airport, (3.8) gives the same argument with one initial unit. Membership in $\mathcal{F}_k^{\downarrow}(t_i^{\uparrow})$ guarantees the turn time. Distinct departure times ensure that every earlier departure is already subtracted, so one inventory unit cannot support two departures. □

3.5 Equal-time departure counterexample

The distinct-event condition is not guaranteed by a timetable. Because (3.2) uses a strict inequality, two flights leaving the same airport at the same time do not appear in each other's departure sets. Both can consume the same aircraft inventory.

Example 3.1 (Algebraically feasible but physically impossible)**.** *Let one aircraft j start at B and let $\Delta t = 30$ minutes. Consider*

$$\begin{aligned} &i_0 : B \to A, && t^{\uparrow}_{i_0} = 500, && t^{\downarrow}_{i_0} = 550, \\ &i_1 : A \to B, && t^{\uparrow}_{i_1} = 600, && t^{\downarrow}_{i_1} = 700, \\ &i_2 : A \to C, && t^{\uparrow}_{i_2} = 600, && t^{\downarrow}_{i_2} = 700. \end{aligned}$$

Set $x_{i_0 j} = x_{i_1 j} = x_{i_2 j} = 1$. Flight i_0 uses the initial unit at B. For i_1 at A,

$$\mathcal{F}^{\downarrow}_{\mathrm{A}}(600) = \{i_0\}, \qquad \mathcal{F}^{\uparrow}_{\mathrm{A}}(600) = \varnothing,$$

because i_0 departs from B and i_2 departs at 600, not strictly before 600. Thus C2 gives $1 - 0 \geq 1$. The calculation for i_2 is identical because i_1 is also excluded from its strict departure set. All compact constraints are satisfied, although one aircraft cannot operate i_1 and i_2 at the same time.

The defect is structural rather than cost-dependent. Maintenance constraints also cannot repair duplicated use of an aircraft. Physical non-overlap must be enforced in the ground model.

3.6 Conflict graph and corrected feasibility model

Associate with flight i the occupied interval

$$I_i = [t^{\uparrow}_i, t^{\downarrow}_i + \Delta t). \tag{3.10}$$

The half-open convention allows a following flight to depart exactly when the required turn ends. Define the set of unordered conflicting pairs

$$\mathcal{O} = \Big\{\{i_1, i_2\} \subseteq \mathcal{F} : I_{i_1} \cap I_{i_2} \neq \varnothing\Big\}. \tag{3.11}$$

For every conflict and aircraft, impose

$$x_{i_1 j} + x_{i_2 j} \leq 1, \qquad \forall \{i_1, i_2\} \in \mathcal{O},\ \forall j \in \mathcal{P}. \tag{3.12}$$

These constraints exclude simultaneous flights, overlapping legs, and connections with insufficient turn time. Some rows duplicate restrictions already implied by C2–C3, but the complete conflict set gives a direct and auditable statement of single-aircraft non-overlap.

Proposition 3.2 (Physical feasibility of the corrected model)**.** *Under this chapter's assumptions, every integral solution satisfying* (3.6)–(3.9) *and* (3.12) *yields one temporally*

ordered, airport-continuous sequence for each aircraft.

Proof. Fix aircraft j. Constraint (3.12) makes the occupied intervals of its assigned flights pairwise disjoint, giving an unambiguous temporal order with the required turn separation. Process that order. At a non-initial airport, C2 requires an unused earlier arrival at the departure airport. At the initial airport, C3 supplies one initial unit and thereafter requires return arrivals. Since assigned occupied intervals do not overlap, an earlier aircraft movement cannot support concurrent continuations. Every departure is therefore connected to the initial position or to a preceding turn-feasible arrival at the same airport. □

The corrected model comprises objective (3.5), C1–C3, C4′, and integrality. It changes neither the assignment variables nor the objective; it removes assignments that cannot represent a physical route.

3.7 Model size and computational implications

The original model has $n_f n_p$ binary variables, n_f coverage equations, and at most one continuity row per flight–aircraft pair. Its row count is $O(n_f n_p)$. This compares favourably with the $O(n_f^2 n_p)$ connection-variable structure of **?**]; **?**] also report fewer variables and constraints than the compared time-space formulation on their instances.

The correction adds $|\mathcal{O}|n_p$ inequalities, producing

$$O\Big(n_f n_p + |\mathcal{O}|n_p\Big) \tag{3.13}$$

rows. In the worst case $|\mathcal{O}| = O(n_f^2)$, so the corrected row count is not universally linear. In realistic timetables, a flight conflicts only with flights whose occupied intervals intersect its own; a sweep over flights ordered by departure time generates these pairs efficiently. The model remains compact in its binary-variable space.

The pairwise inequalities may also strengthen the LP relaxation. Clique inequalities could be generated for sets of mutually overlapping flights, but the pairwise form is retained because it is transparent, easy to validate, and sufficient for integral physical feasibility.

3.8 Illustrative non-cyclic assignment

Table 3.2 reproduces the structure of the six-flight example in **?**]. The turn time is 30 minutes; aircraft 1 starts at A and aircraft 2 at B.

Table 3.2: Six-flight timetable for the illustrative ground assignment.

Flight	Departure	Arrival	Departure time	Arrival time
1	A	B	09:00	11:00
2	B	A	11:30	13:30
3	A	B	14:00	16:00
4	B	A	09:30	11:30
5	A	B	12:00	14:00
6	B	A	14:30	16:30

One feasible assignment is

$$\text{aircraft 1: } 1 \to 2 \to 3,$$
$$\text{aircraft 2: } 4 \to 5 \to 6.$$

Every connection preserves airport continuity and uses exactly the 30-minute turn. Aircraft 1 ends at B and aircraft 2 at A, so the assignment is feasible for the non-cyclic horizon. Requiring both aircraft to return to their initial airports would reject it, illustrating that cyclicity is a terminal policy and not a requirement for physical feasibility within the horizon.

3.9 Chapter summary

The compact model obtains a small assignment-variable space by replacing explicit connections with cumulative airport balances. Those balances encode initial position, continuity, and turn time when departures are strictly ordered. Equal-time departures expose a missing case because two flights can consume the same inventory. The conflict inequalities (3.12) close this gap and establish the physically executable domain used by the maintenance model in Chapter 4.

Chapter 4

Maintenance-Extended Model and the Correction of Constraint (13)

Building on the corrected ground model of Chapter 3, this chapter incorporates maintenance feasibility into the compact tail-assignment formulation. The assignment variables determine where each aircraft is during the planning horizon, but they do not state whether a required check can be performed at that location or whether the aircraft has exceeded its allowable exposure since the preceding check. The maintenance layer therefore couples three decisions: the flight after which a check is initiated, the day on which the check is recorded, and the cumulative utilisation that the check resets.

The presentation begins with one hours-based type-A check, following the maintenance extension of **?**]. This isolates the logic of the cumulative-hour restriction and the correction of Constraint (13). The four-level A/B/C/D hierarchy, check subsumption, and multi-day heavy checks are introduced in Chapter 5.

4.1 Scope and operational assumptions

The maintenance model inherits all flight coverage, airport continuity, turn time, and non-overlap assumptions of Chapter 3. In addition, it uses the following assumptions.

1. A type-A check is required before cumulative block time exceeds the threshold $T^A_{\max}$.
2. A check can be initiated only after an aircraft arrives at a maintenance-capable airport.
3. A check consumes one unit of airport maintenance capacity on its scheduled day.
4. In the single-check model, at most one check is performed on an aircraft on a given day.

5. A triggered overnight check prevents the aircraft from operating a later flight on the same day. Longer check durations are treated in Chapter 5.

6. The state entering the horizon is known: the initial airport and the cumulative hours Φ_j^A since the last type-A check are input data.

The day index records maintenance opportunities; the flight timetable remains continuous in absolute time. Consequently, an arrival must be both spatially eligible and temporally compatible with a check. Merely ending a day at an arbitrary airport is insufficient.

4.2 Sets, parameters, and auxiliary definitions

Let $\mathcal{D} = \{1, \dots, n_d\}$ be the ordered set of planning days and $\mathcal{M} \subseteq \mathcal{A}$ the set of maintenance-capable airports. For each flight i, let $d_i^{\uparrow}$ and $d_i^{\downarrow}$ be its departure and arrival days, $\tilde{t}_i$ its block time, and $a_i^{\downarrow}$ its arrival airport. Define

$$\mathcal{F}^{\downarrow}(m) = \{i \in \mathcal{F} : a_i^{\downarrow} = m\}, \qquad m \in \mathcal{M}, \tag{4.1}$$

and let

$$\mathcal{F}^{+}(d, i) = \{i' \in \mathcal{F} : d_{i'}^{\uparrow} = d,\ t_{i'}^{\uparrow} > t_i^{\downarrow}\} \tag{4.2}$$

be the flights departing later on day d than the arrival of flight i. Only departures that are otherwise compatible with aircraft j need to be included when the inequalities are generated.

For $d < d'$, denote by

$$F_{d,d'} = \{i \in \mathcal{F} : d < d_i^{\uparrow} < d'\} \tag{4.3}$$

the flights whose departure days lie strictly inside the day pair. This open-interval convention separates flights accumulated between the two boundary-day reset decisions. Other endpoint conventions are possible, but the set definition and reset semantics must be used consistently.

The additional parameters are summarised in Table 4.1.

4.3 Maintenance decision variables

For every maintenance-eligible arrival, aircraft, and feasible check day, define

$$z_{ijd} = \begin{cases} 1, & \text{if aircraft } j \text{ initiates a type-A check after flight } i \\ & \text{and the check is recorded on day } d, \\ 0, & \text{otherwise.} \end{cases} \tag{4.4}$$

Table 4.1: Additional notation for the type-A maintenance model.

Symbol	Meaning
$\mathcal{D} = \{1, \dots, n_d\}$	Ordered set of planning days
$\mathcal{M} \subseteq \mathcal{A}$	Set of maintenance-capable airports
mcap_{md}	Number of check slots available at airport m on day d
mc_{ijd}	Cost of checking aircraft j after flight i on day d
$T^A_{\max}$	Maximum block time accumulated without a type-A check
$d^A_{\max}$	Maximum admissible calendar spacing used to generate check windows
Φ^A_j	Type-A block time accumulated by aircraft j before the horizon
$M_{dd'}$	Valid big-M value for the day pair (d, d')

The variable is created only when $i \in \mathcal{F}^{\downarrow}(m)$ for some $m \in \mathcal{M}$ and when day d is compatible with the arrival and check duration. This sparse domain is preferable to creating impossible variables and fixing them to zero.

The daily aggregate variable is

$$y_{jd} = \begin{cases} 1, & \text{if aircraft } j \text{ undergoes a type-A check on day } d, \\ 0, & \text{otherwise.} \end{cases} \tag{4.5}$$

The distinction is important: z_{ijd} identifies the arrival and therefore the maintenance airport, whereas y_{jd} is the reset indicator used in the spacing and cumulative-hour constraints.

4.4 Extended objective function

The basic assignment objective is augmented by the cost of selected maintenance events:

$$\min \quad \sum_{i \in \mathcal{F}} \sum_{j \in \mathcal{P}} c_{ij} x_{ij} + \sum_{m \in \mathcal{M}} \sum_{i \in \mathcal{F}^{\downarrow}(m)} \sum_{j \in \mathcal{P}} \sum_{d \in \mathcal{D}} mc_{ijd} z_{ijd}. \tag{4.6}$$

The coefficient mc_{ijd} may combine labour, station usage, overnight parking, and opportunity cost. A maintenance-ineligible assignment must be excluded by the variable domain or constraints; a large objective coefficient does not make it infeasible.

4.5 Maintenance-event feasibility

4.5.1 C8: blocking subsequent same-day flights

If maintenance begins after flight i, aircraft j cannot operate a later flight while the check is active. For the overnight type-A abstraction this is written as

$$z_{ijd} + x_{i'j} \leq 1, \qquad \begin{smallmatrix} \forall i \in \mathcal{F}^{\downarrow}(m),\ m \in \mathcal{M}, \\ \forall i' \in \mathcal{F}^{+}(d,i),\ j \in \mathcal{P},\ d \in \mathcal{D}. \end{smallmatrix} \tag{4.7}$$

When the precise check duration is available, $\mathcal{F}^{+}(d,i)$ can be replaced by the departures whose times intersect the maintenance interval. The essential implication remains the same: selecting z_{ijd} removes every flight that would make the check physically impossible.

4.5.2 C9: coupling a check to its trigger flight

A check can be attached to flight i only if that flight is assigned to the same aircraft:

$$z_{ijd} \leq x_{ij}, \qquad \forall (i,j,d) \text{ in the domain of } z. \tag{4.8}$$

If several feasible check days are associated with the same arrival, the stronger aggregated form $\sum_d z_{ijd} \leq x_{ij}$ prevents that arrival from triggering more than one check.

4.5.3 C10: airport-day maintenance capacity

All checks initiated at airport m on day d share its finite capacity:

$$\sum_{i \in \mathcal{F}^{\downarrow}(m)} \sum_{j \in \mathcal{P}} z_{ijd} \leq \mathrm{mcap}_{md}, \qquad \forall m \in \mathcal{M},\ d \in \mathcal{D}. \tag{4.9}$$

This is a resource constraint rather than an aircraft-feasibility constraint: each selected trigger can be individually valid while their simultaneous use of the same station is infeasible.

4.5.4 C11 and C14: daily aggregation and uniqueness

The trigger and reset representations are linked by

$$\sum_{m \in \mathcal{M}} \sum_{i \in \mathcal{F}^{\downarrow}(m)} z_{ijd} = y_{jd}, \qquad \forall j \in \mathcal{P},\ d \in \mathcal{D}. \tag{4.10}$$

Because y_{jd} is binary, equality (4.10) already permits at most one trigger for aircraft j on day d. The explicit inequality

$$\sum_{m\in\mathcal{M}}\sum_{i\in\mathcal{F}^{\downarrow}(m)} z_{ijd} \le 1, \qquad \forall j\in\mathcal{P},\ d\in\mathcal{D}, \tag{4.11}$$

is therefore redundant in the single-type model but documents the operational rule and becomes useful when several check types share a day.

4.5.5 C12: maximum calendar spacing

For every window containing $d^A_{\max}$ consecutive days, at least one check must occur:

$$\sum_{r=d}^{d+d^A_{\max}-1} y_{jr} \ge 1, \qquad \substack{\forall j\in\mathcal{P},\\ d=1,\dots,n_d-d^A_{\max}+1.} \tag{4.12}$$

This sliding-window form is equivalent to requiring that no run of $d^A_{\max}$ days is check-free. It limits calendar spacing, but it cannot replace a flight-hour constraint: an aircraft may accumulate the hour limit early in a permissible calendar window.

4.6 Original cumulative-hour constraint

For aircraft j and day pair (d, d'), define

$$H_{jdd'} = \sum_{i\in F_{d,d'}} \tilde{t}_i x_{ij}, \qquad Y^{\circ}_{jdd'} = \sum_{r=d+1}^{d'-1} y_{jr}. \tag{4.13}$$

Here $Y^{\circ}_{jdd'}$ counts checks strictly inside the interval. The original formulation of **?**] uses the single inequality

$$H_{jdd'} \le T^A_{\max} + M_{dd'}(2 - y_{jd} - y_{jd'}) + M_{dd'}Y^{\circ}_{jdd'}. \tag{4.14}$$

It is generated for the admissible day pairs, typically $2 \le d' - d \le d^A_{\max} - 1$.

The intended logic is to enforce the hour limit on an interval without an intervening reset and to relax it when a check splits the accumulation period. The endpoint expression does not implement that logic for all endpoint patterns. If $Y^{\circ}_{jdd'} = 0$ and exactly one endpoint has a check, then (4.14) becomes

$$H_{jdd'} \le T^A_{\max} + M_{dd'}, \tag{4.15}$$

which is non-binding for a valid big-M. If neither endpoint has a check, the available slack increases to $2M_{dd'}$. Thus the original row controls only the endpoint pattern

$(y_{jd}, y_{jd'}) = (1, 1)$ when the interior is check-free.

Lemma 4.1 (Endpoint activation defect)**.** *Suppose $Y^{\circ}_{jdd'} = 0$ and $M_{dd'}$ is at least the maximum attainable value of $H_{jdd'}$. If either endpoint indicator is zero, Constraint* (4.14) *cannot exclude an assignment with $H_{jdd'} > T^{A}_{\max}$.*

Proof. If exactly one endpoint is zero, the right-hand side is $T^{A}_{\max} + M_{dd'}$; if both are zero, it is $T^{A}_{\max} + 2M_{dd'}$. In either case it is no smaller than every attainable value of $H_{jdd'}$, so the row cannot enforce the threshold. □

For example, let $T^{A}_{\max} = 60$, $M_{12} = 200$, and let the assigned flights accumulate 150 minutes over the interval. With $(y_{j1}, y_{j2}) = (0, 1)$ and no interior check, the original inequality reads $150 \leq 260$ and accepts the overrun.

4.7 Corrected endpoint-anchored formulation

The correction replaces the combined endpoint expression by two inequalities, one anchored at each boundary:

$$H_{jdd'} \leq T^{A}_{\max} + M_{dd'} Y^{\circ}_{jdd'} + M_{dd'} y_{jd}, \tag{4.16}$$

$$H_{jdd'} \leq T^{A}_{\max} + M_{dd'} Y^{\circ}_{jdd'} + M_{dd'} y_{jd'}. \tag{4.17}$$

The pair is generated for every aircraft and every admissible day pair. The first row is relaxed by a check at the start boundary; the second is relaxed by a check at the end boundary. An interior check relaxes both because the interval then contains a reset and must be analysed through shorter windows.

Table 4.2 makes the binary endpoint logic explicit for an interval with no interior check.

Table 4.2: Activation of the original and corrected C13 rows when $Y^{\circ}_{jdd'} = 0$. "Active" means that $H_{jdd'} \leq T^{A}_{\max}$.

y_{jd}	$y_{jd'}$	Original row	Corrected pair
0	0	Relaxed by $2M_{dd'}$	Both active
0	1	Relaxed by $M_{dd'}$	Start-anchored row active
1	0	Relaxed by $M_{dd'}$	End-anchored row active
1	1	Active	Both endpoint rows relaxed

The table also clarifies the modelling convention behind the correction. Endpoint indicators act as reset relaxers for their respective anchored rows; the corrected pair is designed to control check-free windows and one-sided opening or closing windows. It should not be justified by claiming that each new row dominates the original row for

every fixed binary endpoint pattern. Its validity follows from generating all admissible windows together with C12 and the opening-horizon condition below.

Proposition 4.1 (Check-free-window enforcement)**.** *For any admissible interval with no interior check and at least one check-free endpoint, Constraints* (4.16)–(4.17) *impose* $H_{jdd'} \leq T^A_{\max}$.

Proof. If $y_{jd} = 0$, the big-M endpoint term vanishes from (4.16); if $y_{jd'} = 0$, it vanishes from (4.17). Since the interior sum is zero, either applicable row reduces to $H_{jdd'} \leq T^A_{\max}$. If both endpoints are zero, both rows reduce to that inequality. □

The numerical example above is rejected by the start-anchored row:

$$150 \leq 60 + 200(0) + 200(0) = 60, \tag{4.18}$$

which is false. The correction therefore closes precisely the one-endpoint case admitted by the original formulation.

4.8 Pre-horizon accumulated hours

Without an opening condition, an aircraft entering the horizon close to its limit is incorrectly treated as newly maintained. Let $F_{0,d'}$ contain the flights departing strictly before boundary day d'. The remaining allowance before the first in-horizon check is $T^A_{\max} - \Phi^A_j$, giving

$$\sum_{i \in F_{0,d'}} \tilde{t}_i x_{ij} \leq T^A_{\max} - \Phi^A_j + M_{0d'} \sum_{r=1}^{d'-1} y_{jr} + M_{0d'} y_{jd'}, \qquad \forall j \in \mathcal{P},\ d' \in \mathcal{D}. \tag{4.19}$$

Before the first check, all y terms vanish and the residual allowance is enforced. Once a check occurs, the pre-horizon counter is reset and the row is relaxed. This constraint is essential whenever $\Phi^A_j > 0$; omitting it can make an apparently feasible schedule violate the true maintenance state on its first day.

4.9 Choice of the big-M coefficient

A global constant equal to the total duration of every flight is valid but unnecessarily weak. The formulation instead uses a day-pair-specific value satisfying

$$M_{dd'} \geq \max \left\{ \sum_{i \in F_{d,d'}} \tilde{t}_i x_{ij} : x \text{ satisfies the corrected ground model for aircraft } j \right\}. \tag{4.20}$$

A safe inexpensive bound is $\sum_{i \in F_{d,d'}} \tilde{t}_i$. A tighter value can be obtained as a longest feasible path in the acyclic flight-connection graph restricted to the interval. Because every $M_{dd'}$ appears directly in the LP relaxation, reducing it without sacrificing validity strengthens the formulation and improves numerical conditioning.

4.10 Physical interpretation and model validity

The maintenance equations do not repair an infeasible flight trajectory; they assume the corrected ground domain of Chapter 3. In particular, Constraint (3.12) ensures that cumulative hours are measured on a trajectory that one aircraft can actually execute. The effect of the ground correction on C8–C14 is therefore a domain correction: the algebra of the maintenance rows is unchanged, but those rows are no longer evaluated on simultaneous or overlapping flight assignments.

Theorem 4.1 (Feasibility of the single-type maintenance layer)**.** *Assume that the admissible C12–C13 windows cover every possible check-free accumulation span. Any integral solution satisfying the corrected ground model, Constraints* (4.7)– (4.12)*, the endpoint pair* (4.16)– (4.17)*, the opening condition* (4.19)*, and the variable domains induces physically executable aircraft trajectories with capacity-feasible type-A maintenance and no cumulative-hour overrun.*

Proof. Fix aircraft j. The result of Chapter 3 orders its assigned flights into a turn-feasible, non-overlapping trajectory. By (4.8), each selected trigger belongs to that trajectory; by (4.7), no later same-day assignment conflicts with the selected check. Equations (4.9) and (4.10) enforce station capacity and a consistent daily reset. Constraint (4.12) prevents an excessive calendar gap.

Before the first in-horizon check, (4.19) includes the entering exposure Φ_j^A. After a reset, suppose that the aircraft accumulated more than $T_{\max}^A$ without another check. By the stated window-coverage assumption, this span contains an admissible interval with no interior check and at least one check-free endpoint. Proposition 4.1 bounds that interval by $T_{\max}^A$, a contradiction. Thus the maintenance schedule is executable and respects the cumulative-hour limit. □

4.11 Complete single-type formulation and size

The maintenance-extended model consists of the corrected ground objective and constraints, objective (4.6), C8–C12 and C14, the corrected C13 pair, and the opening-horizon rows, with

$$x_{ij} \in \{0,1\}, \qquad z_{ijd} \in \{0,1\}, \qquad y_{jd} \in \{0,1\}. \tag{4.21}$$

With sparse trigger indexing, the model adds at most $O(|\mathcal{F}||\mathcal{P}||\mathcal{D}|)$ trigger variables, $O(|\mathcal{P}||\mathcal{D}|)$ daily variables, and $O(|\mathcal{P}||\mathcal{D}|^2)$ cumulative-hour rows. The day-pair rows are the principal growth relative to the $O(|\mathcal{F}||\mathcal{P}|)$ ground assignment layer.

This chapter establishes the maintenance logic for one hours-based check. The next chapter replaces y_{jd} by type-specific and hierarchy-aware reset indicators, distinguishes hours-based A/B checks from calendar-based C/D checks, and represents checks whose duration extends over multiple days.

Chapter 5

Full A/B/C/D Maintenance Hierarchy

The maintenance-extended model of Chapter 4 captures the basic logic of an aircraft-hours check, but not the full regulatory structure of modern airline operations. In practice, maintenance requirements are organized as an ordered hierarchy of four check levels, denoted A, B, C, and D, each with distinct trigger mechanisms, durations, and reset semantics. This chapter extends the compact assignment formulation to the full A/B/C/D regime while retaining the key modelling principle of Chapter 4: a schedule remains compact and assignment-based, with maintenance decisions represented through binary indicators rather than explicit route enumeration.

The extension is not merely a bookkeeping exercise. If the model treats all checks as equivalent or ignores the fact that a heavier check covers lighter ones, it can certify schedules that violate the required maintenance cadence or overstate the amount of slack available between inspections. The hierarchy is therefore introduced through a set of dominance relationships and check-reset indicators that preserve the correct operational semantics while keeping the model tractable.

5.1 Application representation of the hierarchy

In the application, the hierarchy is represented directly in the JSON instance and then loaded by the Python model builder. The instance contains the aircraft set, flight legs and their departure and arrival times, initial aircraft locations, initial maintenance exposure, maintenance thresholds, maintenance durations, station capacities, and the flight–aircraft cost matrix. The keys for the four check levels are interpreted consistently throughout the application: A and B thresholds are measured in accumulated flight minutes, whereas C and D thresholds are measured in elapsed calendar days. Maintenance durations remain expressed in minutes because they consume time in the

daily aircraft timeline.

This data flow creates a clear separation between instance data and model logic. The input file specifies the regulatory and operational state at the start of the planning horizon; `MILP_Sheduler` constructs the binary variables and maintenance constraints; and the resulting solution reports aircraft assignments and maintenance decisions for the selected horizon. The same instance format is consumed by `Scheduler`, the heuristic engine, which maintains the corresponding A/B flight-minute counters and C/D day counters while constructing routes. Consequently, exact and heuristic runs are comparable because they start from the same fleet state and apply the same check hierarchy, even though one represents decisions algebraically and the other evaluates them through timeline simulation.

At application level, a maintenance check is not entered as an arbitrary independent event after optimization. Its type, location, duration, and effect on the aircraft state determine which subsequent flights are feasible. A heavier check is therefore propagated through the hierarchy both in the MILP variables and in the simulator: a D-check resets all four obligations, a C-check resets A, B, and C, a B-check resets A and B, and an A-check resets only A. This shared interpretation is what allows the application to produce not only an assignment matrix, but an operationally interpretable aircraft schedule.

5.2 Check types and regulatory hierarchy

Let

$$\mathcal{C} = \{A, B, C, D\} \tag{5.1}$$

be the ordered set of maintenance classes, with dominance order

$$D \succ C \succ B \succ A. \tag{5.2}$$

A check of type c is said to satisfy all lighter checks $c' \prec c$ in the same period. This means that a D-check resets the elapsed-day requirement for C, and simultaneously satisfies the B- and A-check conditions; similarly, a C-check covers the lighter levels below it. The hierarchy therefore allows the model to represent a single maintenance event as a check of level c while implicitly carrying the obligations of all lower levels that are dominated by that event.

For each aircraft $j \in \mathcal{P}$, day $d \in \mathcal{D}$, and check type $c \in \mathcal{C}$, define

$$y_{jdc} = \begin{cases} 1, & \text{if aircraft } j \text{ undergoes a check of type exactly } c \text{ on day } d, \\ 0, & \text{otherwise.} \end{cases} \tag{5.3}$$

The exact-type variable is complemented by the hierarchy indicator

$$\eta_{jdc} = \begin{cases} 1, & \text{if aircraft } j \text{ undergoes a check of level at least } c \text{ on day } d, \\ 0, & \text{otherwise.} \end{cases} \tag{5.4}$$

The indicator is a cumulative summary of the maintenance event: it is equal to one whenever a check of level at least c is scheduled for aircraft j on day d. In the compact model, we enforce the relation

$$\eta_{jdc} = \sum_{c' \succeq c} y_{jdc'}, \qquad \forall j \in \mathcal{P},\ d \in \mathcal{D},\ c \in \mathcal{C}, \tag{5.5}$$

which ensures that a heavier check automatically implies the lighter reset state for that same day.

This construction is essential. It prevents a schedule from declaring an aircraft as meeting the A-check requirement while it is actually undergoing a heavier event that should satisfy the same obligation. The hierarchy is not a separate planning layer; rather, it is the mechanism through which the model maintains regulatory consistency across all maintenance classes.

5.3 Hierarchy constraints

The dominance structure is encoded through a small set of linear constraints. First, at most one check level may be performed for a given aircraft on a given calendar day:

$$\sum_{c \in \mathcal{C}} y_{jdc} \leq 1, \qquad \forall j \in \mathcal{P},\ d \in \mathcal{D}. \tag{5.6}$$

This operational rule is consistent with the daily maintenance regime in which an aircraft is either free, or engaged in a single maintenance event, but not both.

Second, the hierarchy indicator must respect subsumption. If $c' \succ c$, then

$$\eta_{jdc} \geq \eta_{jdc'}, \qquad \forall j \in \mathcal{P},\ d \in \mathcal{D},\ c \prec c'. \tag{5.7}$$

The inequality states that a heavier event on day d implies the aircraft is also counted as having undergone all lighter checks on that day. Consequently, the same maintenance action simultaneously satisfies the reset condition for all dominated check types.

Finally, the exact-level and cumulative-level variables are tied together by

$$\eta_{jdA} = y_{jdA} + y_{jdB} + y_{jdC} + y_{jdD}, \tag{5.8}$$

$$\eta_{jdB} = y_{jdB} + y_{jdC} + y_{jdD}, \tag{5.9}$$

$$\eta_{jdC} = y_{jdC} + y_{jdD}, \tag{5.10}$$

$$\eta_{jdD} = y_{jdD}, \tag{5.11}$$

which are the instantiated forms of the general relation above. These equalities are computationally convenient because they admit a sparse and transparent definition of the maintenance state without introducing unnecessarily large sets of independent variables.

5.4 Flight-hour checks: types A and B

Types A and B are triggered by cumulative block time. Let $T^c_{\max}$ denote the allowable accumulation limit for check type $c \in \{A, B\}$. The corrected endpoint-anchored formulation of Chapter 4 extends directly to the multi-type case by replacing the single reset variable y_{jd} with the relevant hierarchical indicator η_{jdc}.

For a candidate interval (d, d') and a check type $c \in \{A, B\}$,

$$\sum_{i \in F_{d,d'}} x_{ij}\, \tilde{t}_i \le T^c_{\max} + M_{dd'}\, \eta_{jdc} + M_{dd'} \sum_{r=d+1}^{d'-1} \eta_{jrc}, \tag{5.12}$$

$$\sum_{i \in F_{d,d'}} x_{ij}\, \tilde{t}_i \le T^c_{\max} + M_{dd'}\, \eta_{jd'c} + M_{dd'} \sum_{r=d+1}^{d'-1} \eta_{jrc}, \tag{5.13}$$

where $F_{d,d'}$ is the set of flights departing between the two boundary days. The logic of the correction is retained: a reset at either endpoint or any intermediate day absorbs the flight-hour exposure, and the model does not falsely permit the cumulative hours to exceed the threshold when a required check should have intervened.

This generalization is the key structural step from a single maintenance class to a full regulatory regime. A type-A or type-B reset is not only activated by a same-level event; it may also be triggered by a heavier check, because the heavier event satisfies the lighter requirement through η_{jdc}. This is precisely what preserves the dominance ordering in the cumulative-hour constraints.

5.4.1 Maximum-spacing requirement

The calendar-spacing logic associated with flight-hour checks is also generalized by the hierarchy variable. For each aircraft j and check type $c \in \{A, B\}$, a maintenance event of level at least c must occur within every admissible window:

$$\sum_{r=d}^{d+d^c_{\max}-1} \eta_{jrc} \ge 1, \qquad \forall j \in \mathcal{P},\ d \in \mathcal{D},\ c \in \{A, B\}. \tag{5.14}$$

The inequality ensures that the cumulative exposure is not only bounded in the interval but also reset before the maximum tolerated calendar spacing is exceeded. In this sense, the hierarchy couples two complementary regulatory dimensions: elapsed time and accumulated usage.

5.5 Calendar-day checks: types C and D

Types C and D are governed by elapsed calendar days rather than cumulative flight hours. For each such check type $c \in \{C, D\}$, let $d^c_{\max}$ denote the maximum admissible interval between two consecutive checks of level at least c. The spacing constraint is then

$$\sum_{r=d}^{d+d^c_{\max}-1} \eta_{jrc} \geq 1, \qquad \forall j \in \mathcal{P},\ d \in \mathcal{D},\ c \in \{C, D\}. \tag{5.15}$$

This rule says that in every window of length $d^c_{\max}$, aircraft j must experience at least one check of level at least c. Equivalently, there can be no unbroken period longer than the maximum allowable interval without a qualifying maintenance intervention.

Unlike the A/B checks, heavy checks are usually multi-day maintenance events. If a type-c inspection begins on day d, it occupies a contiguous maintenance window of length ℓ_c, so the aircraft is grounded on the days

$$d,\ d+1,\ \ldots,\ d+\ell_c-1. \tag{5.16}$$

This is modelled by imposing that a maintenance event of type c scheduled on day d propagates to the following days of the maintenance window:

$$y_{j,d+k,c} \geq y_{jdc}, \qquad \forall j \in \mathcal{P},\ d \in \mathcal{D},\ c \in \{C, D\},\ k = 1, \ldots, \ell_c - 1. \tag{5.17}$$

The effect is operationally realistic: a heavy maintenance action cannot be represented as a single-day reset without also forbidding flights during the requisite continuation of the maintenance period.

To preserve feasibility, the model prevents flights from being assigned to an aircraft while it is within an active maintenance window. If a check of type c begins at day d and lasts ℓ_c days, then no flight departing during the affected interval may be assigned to aircraft j:

$$\sum_{i:\ d_i^{\uparrow}=d+k} x_{ij} \leq 1 - y_{jdc}, \qquad \forall j \in \mathcal{P},\ d \in \mathcal{D},\ c \in \{C, D\},\ k = 0, \ldots, \ell_c - 1. \tag{5.18}$$

This is the operational counterpart of the exact-type maintenance variable and prevents the schedule from assigning an aircraft to a flight that would overlap an active heavy

check.

5.6 Pre-horizon state accounting

The maintenance logic remains valid only if the aircraft state at the beginning of the planning horizon is treated consistently. For each aircraft j, let

- Φ_j^A and Φ_j^B be the accumulated flight hours since the preceding A/B check,
- Ψ_j^C and Ψ_j^D be the elapsed calendar days since the preceding C/D check.

These values are not optional modelling extras; they are required to preserve the continuity of regulatory compliance from the previous maintenance cycle into the planning horizon.

Without this carry-forward information, the model implicitly assumes that every aircraft begins the horizon in a fully reset state, which is unrealistic and may cause the generated schedule to violate the maintenance requirements within the first few days of operation. To avoid this, the initial sub-interval is adjusted so that the effective exposure is measured relative to the entry state, rather than assuming a zero-age aircraft at day one. The same principle applies to both hour-based and day-based checks: the first relevant interval in the horizon is shifted by the pre-horizon accumulated value, analogous to the correction already described in Chapter 4 for the single-check model.

This treatment is critical in computational practice because a schedule can appear feasible on the basis of a clean horizon start but still be invalid once the carry-over exposure is restored. The refined formulation therefore maintains operational continuity across planning periods, which is necessary when the observed schedule is intended to be used in a realistic fleet-operations setting.

5.7 Model size and operational implications

The full A/B/C/D extension increases the number of maintenance variables and constraints relative to the basic maintenance model, but it does so in a controlled and structured way. The exact-type variables y_{jdc} are defined for each aircraft, day, and check level, so the additional binary dimension is of order $O(|\mathcal{P}|\,|\mathcal{D}|\,|\mathcal{C}|)$. The hierarchy indicator variables η_{jdc} add the same order of magnitude, while the spacing, dominance, and heavy-check constraints contribute a comparable set of linear rows. The extended formulation therefore remains compact and assignment-based; it does not enumerate complete aircraft routes or repair schedules explicitly.

Proposition 5.1. *The addition of the full $\{A, B, C, D\}$ hierarchy to the maintenance-extended compact model increases the number of maintenance-related binary variables by*

a factor bounded by $|\mathcal{C}| = 4$ and introduces a linear constraint set of comparable order. The model remains polynomial in the size of the daily assignment network and does not require route enumeration.

The operational value of the extension is substantial. A compact formulation that ignores maintenance hierarchy may return schedules that are numerically optimal for the wrong problem: it may overlook a required reset, permit illegal overlapping inspection windows, or certify a solution that fails under real regulatory enforcement. By contrast, the A/B/C/D hierarchy captures the actual regulatory logic used in airline planning and preserves the balance between operational realism and tractability that is central to the thesis.

Chapter 6

Heuristic Solution Methods

Exact optimisation provides a certificate of optimality, but the compact aircraft-routing model becomes computationally demanding on dense or maintenance-heavy instances. For this reason, heuristic methods in the present project serve two roles: they provide fast feasible schedules when the exact solver is not tractable, and they provide good primal solutions that can be used as warm starts for the MILP. The heuristics are therefore designed to preserve the same operational semantics as the exact formulation, especially the hierarchy of checks $D \succ C \succ B \succ A$ and the timeline feasibility rules established in Chapters 4–5.

The current implementation contains constructive, improvement, and network-search procedures in the `Scheduler` class of `src/model.py`. The first group operates directly on ordered flight lists and repeatedly invokes a timeline simulator. The modified Dijkstra heuristic now implemented in the same class uses an implicit time-ordered flight-transition network. The ACO mode uses the same route representation but adds pheromone-guided probabilistic construction and best-ant reinforcement.

6.1 Taxonomy of heuristic methods

The heuristics used in this project can be grouped into three main classes.

1. **Constructive assignment heuristics.** These methods build a schedule from scratch by assigning flights to aircraft in chronological order. The principal representatives are the greedy heuristic and the insertion-based heuristic implemented in the project scheduler.

2. **Repair and local-improvement heuristics.** These methods start from an initial feasible or partial assignment and iteratively improve it by trying insertions or relocations that preserve feasibility. In the current implementation this class includes repair and local-search refinements.

3. **Network-search heuristics.** The modified Dijkstra mode searches implicit time-ordered flight transitions, while ACO samples timeline-feasible transitions using pheromone and cost desirability.

The concrete modes accepted by `Scheduler._normalize_heuristic` are summarized below.

Mode	Construction	Improvement phase
`greedy`	append to cheapest feasible aircraft route	none
`insertion`	best feasible position over all routes	none
`greedy+insertion`	greedy append	up to five insertion sweeps
`repair`	greedy append	up to eight insertion sweeps
`local_search`	greedy append	relocation, then up to four ins
`dijkstra`	maximum-coverage label search	repeated path selection
`aco`	pheromone-guided chronological construction	best-ant reinforcement

Thus, the implementation combines route-list procedures with two implicit graph-search procedures. The shared feasibility simulator is the controlling abstraction; the constructive modes modify ordered route lists directly, whereas Dijkstra uses labels and ACO uses pheromone-guided route sampling.

6.2 Shared feasibility oracle

All heuristic procedures rely on a common feasibility oracle that evaluates an aircraft route in chronological order. Given an ordered flight list $[f_1, \ldots, f_k]$ for aircraft j, the simulator returns either a feasible event sequence with total accumulated cost, or an infeasible status. This is the central primitive of the project scheduler and it embodies the same maintenance logic used in the exact model.

For each aircraft, the simulator maintains four running counters: the elapsed flight minutes since the last A- or B-check, and the elapsed calendar-day counters since the last C- or D-check. Each new flight is accepted only if the aircraft can reach the required departure airport in time, tolerate any necessary repositioning, and satisfy maintenance-trigger conditions before the departure time. If a maintenance check is triggered, the simulator checks that the aircraft is at a maintenance-capable origin airport, that the required station capacity is available, and that the check duration can be completed before the next flight departure. The aircraft is then reset according to the dominance hierarchy $D \succ C \succ B \succ A$. A route is rejected immediately if any of these conditions fail.

This design is deliberate. A flight assignment can appear locally attractive at the cost level while still violating the operational sequence of the schedule. For example, an aircraft might be assigned a new flight too soon after a previous departure, leaving insufficient time for a required maintenance check, or it might accumulate flight hours beyond the permitted threshold before the reset is applied. The timeline oracle filters out these invalid sequences before the assignment is accepted.

In the current implementation, the main logic is encoded in the class `Scheduler` and the method `get_timeline`. The function is the concrete realisation of the abstract feasibility check that is also used in the network-based algorithms: whenever an aircraft route is extended by another flight, the simulator is called to confirm that the maintained route remains feasible with respect to turn times, maintenance durations, capacities, and reset rules.

6.3 Greedy construction

The most direct constructive heuristic is a chronological greedy assignment strategy. Flights are ordered by departure time and processed one by one. For the current flight i, the algorithm considers every aircraft j and evaluates the extended route $R_j \cup \{i\}$ using the timeline oracle. Among the feasible options it selects the aircraft that yields the minimum incremental assignment cost:

$$j^* = \arg \min_{j \in \mathcal{P}^{\text{feas}}(i)} c_{ij}, \tag{6.1}$$

where $\mathcal{P}^{\text{feas}}(i)$ denotes the aircraft for which the new route remains feasible under the timeline rules. If no aircraft can accommodate flight i, the flight remains unassigned and the search continues with the next flight.

This method is simple, deterministic, and fast. It is also highly transparent, which makes it a natural baseline in computational experiments. The implementation exposes this strategy as the `greedy` mode. The route is extended only at its current end: the candidate sequence is `ac_fids[aid] + [fid]`, and the complete candidate timeline is simulated before acceptance. If no aircraft can accommodate a flight, that flight is placed in the unassigned set rather than being forced into an invalid route.

The strength of the greedy scheme is its speed. The weakness is its myopia. A locally cheapest assignment may block later flights or create a maintenance state that leaves no feasible schedule for the remaining network. This is particularly relevant in dense instances where the feasible room for each aircraft is tightly constrained by the maintenance thresholds and by the aircraft-specific departure and arrival times. In such cases, greediness can produce a complete schedule only for a subset of the flights, or can concentrate aircraft in a way that is cost-efficient locally but poor globally.

6.4 Insertion-based construction and repair

The project's insertion routine is implemented by `_best_feasible_insertion`. For a candidate flight i, it enumerates every aircraft j and every position $q \in \{0, \ldots, |R_j|\}$ in the current ordered route R_j. It constructs the trial route

$$R_j^{(q,i)} = (R_{j,1}, \ldots, R_{j,q}, i, R_{j,q+1}, \ldots, R_{j,|R_j|}) \tag{6.2}$$

and accepts the feasible candidate with minimum simulated route cost. The `insertion` mode applies this procedure directly to flights in chronological order; the other modes use it as a repair phase for flights that remain unassigned.

This refinement is especially effective when the greedy pass leaves some flights unassigned because the aircraft state is too constrained at the moment of first selection. Insertion repair differs fundamentally from a pure greedy assignment: it does not only ask whether a flight can be added at the end of a route, but whether there exists some point in the current route where the new sequence becomes feasible. As a result, insertion repair can recover flights that were initially rejected due to local incompatibility with the current partial schedule.

The `greedy+insertion` mode performs a greedy pass followed by five insertion sweeps. The `repair` mode performs the same greedy pass and then permits up to eight sweeps, stopping early when a sweep inserts no flight. These iteration limits are part of the implemented algorithm and make the repair effort bounded and reproducible.

The `local_search` mode adds a relocation neighbourhood. For every currently assigned flight i, `_best_feasible_relocation` removes i from its current aircraft and tries every target aircraft and every target position. The move is accepted only when both affected routes remain feasible and the total simulated cost decreases strictly. Three relocation passes are attempted, followed by up to four insertion sweeps for flights still unassigned.

The repair and local-search refinements are therefore not separate conceptual models; they are different levels of the same algorithmic principle. At each step, the algorithm asks whether a small change to the current assignment improves feasibility or decreases cost while respecting the maintenance oracle. This is a standard local-improvement strategy for assignment models with strong temporal and maintenance coupling.

6.5 Ferry and no-ferry variants

The scheduler supports two operational modes through the Boolean parameter `allow_ferry`, also exposed by the command-line option `-no-ferry`. In the default ferry-enabled mode, an aircraft may be repositioned from its current airport k to the origin o_i of the next

flight. The implementation inserts a `FERRY` event of fixed duration $\tau^{\text{ferry}} = 60$ minutes and adds the fixed ferry cost $\kappa^{\text{ferry}} = 6000$ to the route objective. The candidate remains feasible only if

$$t_{\text{current}} + \tau^{\text{ferry}} \leq t_i^{\text{dep}}. \tag{6.3}$$

In the no-ferry mode, a location mismatch is an immediate rejection: if the aircraft finishes at an airport different from the next flight origin, `get_timeline` returns INFEASIBLE. Consequently, the no-ferry heuristic searches only routes whose consecutive flights are physically connected, and it may leave more flights unassigned. The two modes therefore represent different operational assumptions, not merely different penalty settings: ferry-enabled routes may include explicit repositioning events, whereas no-ferry routes must be airport-continuous.

The MILP uses the same switch but has a different modelling interpretation. With `allow_ferry=True`, the model adds the C2–C3 equipment-flow and turnaround constraints, preventing implicit teleportation between incompatible flight endpoints. With `allow_ferry=False`, those constraints are omitted, producing a smaller pure assignment-plus-maintenance model; it does not create explicit ferry variables. Thus, a no-ferry heuristic schedule and a MILP solution built without C2–C3 should not be interpreted as exactly the same operational model.

6.6 Path-based and arc-based representations

The heuristic and MILP use different representations of aircraft movement. The heuristic is *path-list based*: for each aircraft j it stores an ordered sequence

$$R_j = (i_1, i_2, \ldots, i_{q_j}), \tag{6.4}$$

and feasibility is determined by simulating the whole sequence from the aircraft's initial airport and maintenance state. Appending, inserting, or relocating a flight changes a route path and triggers re-evaluation of the affected sequence. There is no explicit node set, arc set, shortest-path label, or flow variable in this heuristic implementation.

The MILP is instead *arc based* at the assignment level. Its sparse binary variables x_{ij} represent assignment arcs from flight i to aircraft j; the set of permitted pairs is precomputed from the cost matrix. Additional temporal and maintenance constraints couple these assignment arcs, flight pairs, and maintenance decisions. The MILP therefore derives route consistency from global constraints rather than storing one mutable route list per aircraft.

This distinction also explains the role of the ferry switch. In the heuristic, ferrying is an explicit event inserted into a path during simulation. In the MILP, ferry-enabled mode is represented indirectly through equipment-flow and turnaround constraints, while

the assignment variables remain flight–aircraft arcs. The project implements Dijkstra and ACO on implicit transition graphs rather than as separate explicit time-expanded network data structures.

6.7 Modified Dijkstra heuristic

A more global alternative to the direct flight-list heuristics is the implemented modified Dijkstra mode. It represents the space-time network implicitly: each candidate flight is a node, and a transition from flight i to flight i' is considered only when the time ordering is valid and the complete trial route passes the timeline simulator. This avoids constructing a separate time-expanded graph while retaining its path-search interpretation.

The network representation is useful because it makes temporal precedence explicit. Before a flight may be scheduled, the algorithm must demonstrate that the aircraft can reach the correct origin airport, complete any necessary maintenance, and satisfy the timing constraints imposed by the remainder of the route. In the implementation, these conditions are not duplicated as arc formulas; they are delegated to `get_timeline`, which validates every candidate label extension.

For each aircraft, the method maintains at most one best label per terminal flight. A label contains the ordered path and its simulated incremental cost. Candidate labels are compared lexicographically: the primary criterion is the number of covered flights, and the secondary criterion is the incremental route cost. This is the modified Dijkstra rule from the project's mathematical formulation: it maximizes covered flight arcs rather than minimizing ordinary distance.

The outer loop repeatedly evaluates the best path available for each aircraft over the currently unassigned flights. It selects the path that covers the largest number of flights and breaks ties by lower incremental cost, appends the path to the selected aircraft route, marks its flights as assigned, and repeats until no feasible path remains. The executable mode is selected with `-heuristic dijkstra`; aliases include `modifieddijkstra` and `dijkstraheuristic`.

Because the candidate transitions are ordered by departure time, the implicit graph is acyclic. The implementation consequently uses a label-setting dynamic-programming traversal rather than a conventional heap-based shortest-path implementation. Its complexity is higher than a single greedy pass because candidate route extensions repeatedly invoke the full timeline simulator, but it can recover longer feasible sequences than a purely local append decision.

6.8 Ant colony optimisation heuristic

The implemented ACO mode constructs a colony of candidate assignments over the flights sorted by departure time. For each flight, an ant considers every aircraft whose route remains feasible after appending the flight. Let p be the previous flight on aircraft j, or a source symbol if the route is empty. The transition trail is $\tau_{j,p,i}$ and the cost desirability is

$$\eta_{j,p,i} = \frac{1}{1 + \max(0, \Delta c_{j,i})}, \tag{6.5}$$

where $\Delta c_{j,i}$ is the incremental timeline cost of assigning flight i to aircraft j. The ant samples an aircraft with probability proportional to

$$\tau_{j,p,i}\, \eta_{j,p,i}^{\beta}. \tag{6.6}$$

Infeasible transitions have zero probability because they are removed before sampling.

After all ants have constructed their assignments, pheromone evaporates toward its initial value $\tau_0 = 1$:

$$\tau_{j,p,i} \leftarrow (1 - \rho)\tau_{j,p,i} + \rho\tau_0, \tag{6.7}$$

where ρ is `aco_evaporation`. The best ant of the iteration then deposits additional pheromone on its visited transitions. Ant quality is ordered first by the number of assigned flights and then by lower total simulated cost. The default implementation uses twenty iterations, eight ants, $\rho = 0.2$, $\beta = 2$, and seed zero; all are configurable through the `Scheduler` constructor.

The mode is selected with `-heuristic aco`. A local pseudo-random generator makes repeated runs reproducible for a fixed seed, while the pheromone updates allow exploration of alternatives to deterministic greedy assignment. The full timeline oracle remains authoritative, so ACO cannot introduce ferry, timing, or maintenance violations even when a probabilistic transition is selected.

6.9 Application workflow and generated schedules

The heuristic algorithms are exposed through the same application entry point as the exact optimizer. A run begins by reading one JSON instance, constructing the `Scheduler`, selecting a heuristic mode, and sorting the flights by departure time. The scheduler then builds one ordered route list per aircraft. At every construction or improvement step, the proposed route is passed to `get_timeline`; only routes that return a feasible event sequence are accepted. This makes the application workflow explicit: the heuristic is not a post-processing visualization of an assignment, but the component that creates and validates the aircraft schedule.

For a command-line run, the principal controls are `-data` for the input instance, `-heuristic` for the selected method, and `-no-ferry` for the airport-continuity variant. The default mode is `greedy+insertion`; the other executable choices are `greedy`, `insertion`, `repair`, `local_search`, `dijkstra`, and `aco`. The ferry switch is applied before maintenance evaluation: with ferrying enabled, a repositioning event must fit before the next departure and contributes its fixed cost; with ferrying disabled, an airport mismatch rejects the candidate route immediately.

The application returns three related views of the same result. The internal route dictionary maps each aircraft to its ordered flight identifiers; the event representation expands each route into flight, ferry, and maintenance events; and the tabular output records aircraft, event type, locations, times, durations, and remaining maintenance margins. The optional Gantt output is a visual projection of these events, while the CSV and event text files provide machine-readable records for subsequent analysis. Unassigned flights are kept explicitly rather than silently discarded, allowing coverage and feasibility to be reported separately.

The heuristic and exact optimizer are exposed through the same application, but their current execution paths remain separate. The `warm_start` argument is retained in `run_milp` for solver-control compatibility; the present `MILP_Sheduler.warm_start_from_heuristic` method is a deliberate no-op because a partial assignment-only start was rejected as infeasible by CPLEX. Thus, the heuristic currently supplies an independent executable schedule and benchmark, while the MILP searches for an improved or certified solution from its own model state. A future complete warm start would need to initialize all coupled assignment, maintenance, and timing variables consistently.

6.10 Quality assessment and computational role

The heuristic methods are evaluated using the same objective as the exact model: the total assignment cost, with penalties for infeasible or unassigned flights. Since the exact MILP provides a lower bound, or even a true optimum on smaller instances, it is natural to assess heuristic quality using an optimality-gap measure. For a heuristic solution with objective value z_{H} and a reference bound z_{ref}, the relative gap is

$$\delta_{\mathrm{H}} = \frac{z_{\mathrm{H}} - z_{\mathrm{ref}}}{z_{\mathrm{ref}}} \times 100\%. \tag{6.8}$$

When the exact MILP is solved to optimality, $z_{\mathrm{ref}} = z^*$; otherwise, z_{ref} can be taken as the LP bound, which still provides a rigorous theoretical benchmark for assessing the quality of the heuristic incumbent. This standard is important because raw cost values are not directly comparable across instances with different sizes and maintenance pressures; the gap measure isolates the quality of the heuristic relative to the underlying

optimisation problem.

The overall conclusion is that the heuristic family is not just a fallback technique. It is a structured collection of methods tailored to the peculiar structure of the tail-assignment problem: heavy temporal coupling, maintenance-trigger logic, and non-trivial interaction between assignment decisions and route feasibility. The deterministic timeline-based methods are the project's main practical tools, while the Dijkstra and ACO formulations remain important as more global search concepts that illuminate the trade-off between solution quality, robustness, and computational effort.

Chapter 7

Computational Study

This chapter evaluates the compact aircraft-routing and maintenance model and the executable heuristic methods introduced in Chapters 4– 6. The computational study has two purposes. First, it checks that the corrected formulation and the A/B/C/D hierarchy behave as intended on controlled boundary cases. Second, it compares the solution quality, coverage, model size, and runtime of exact and heuristic methods under increasing routing and maintenance pressure. The chapter distinguishes measured results from additional comparisons that require a longer computational study.

7.1 Research questions and evaluation principles

The study is organised around the following questions:

1. Does the corrected endpoint-anchored formulation of Constraint (13) preserve feasibility and strengthen the relaxation on maintenance-active instances?
2. How do the number of flights, aircraft, maintenance constraints, and station capacities affect MILP model size, solve status, runtime, and optimality certification?
3. How do the seven executable heuristic modes compare with one another in coverage, objective value, runtime, and route feasibility?
4. How far are heuristic solutions from a certified MILP optimum or, when certification is unavailable, from a rigorous LP lower bound?
5. What is the computational trade-off between the assignment-arc and route/path representations of the same scheduling problem?

The comparison is instance-matched. A method is compared only on the same JSON instance, with the same aircraft initial positions, initial maintenance state, flight costs, maintenance thresholds, station capacities, and ferry setting. The objective values are

interpreted together with coverage: a low cost obtained by leaving flights unassigned is not comparable to a complete assignment unless coverage is reported explicitly. This distinction is essential for the ACO and other partial constructive runs.

7.2 Software, hardware, and reproducibility protocol

The application is implemented in Python using Pyomo for the exact model and pure Python for the heuristic scheduler. The exact runs use a configured MILP backend, principally CPLEX in the archived benchmark results and HiGHS or CBC in the current open-source validation runs. Heuristics require no external optimizer. All runs use the repository entry point `src/model.py`; the legacy `Model_28Oct.py` and earlier standalone heuristic files are not part of the reported implementation.

Each run is defined by the tuple

$$(\text{instance},\ \text{mode},\ \text{solver},\ \text{time limit},\ \text{ferry setting},\ \text{maintenance setting},\ \text{seed}), \tag{7.1}$$

which is recorded with the output files. The random instance generator uses the stable seed function specified in Chapter 1; therefore an instance can be regenerated from its density, fleet size, horizon, and test index. Subsequent comparisons should retain the same instance files and should not replace an archived result merely because a different solver backend produces a more convenient status.

For an exact run, the primary measurements are solver status, objective value, best bound, relative MIP gap, wall-clock time, explored nodes, number of model variables, and number of constraints. For a heuristic run, the primary measurements are assigned flights, unassigned flights, route feasibility, objective value including ferry costs, wall-clock time, and the heuristic parameters. The common output schema used by the batch tools includes `stem`, P, H, F, `vars`, `constraints`, `nodes_max`, `gap_pct`, `cpu_s`, and `objective`; heuristic-specific files additionally record coverage and unassigned counts.

7.2.1 Execution protocol

The single-instance heuristic protocol is:

```
.venv\Scripts\python.exe src\model.py --mode heuristic \\
  --data <instance.json> --heuristic <mode> --no-show
```

The exact protocol is:

```
.venv\Scripts\python.exe src\model.py --mode milp \\
  --data <instance.json> --solver <solver> --time-limit <seconds> --no-show
```

Table 7.1: Provenance classes used for the computational results.

Result class	Typical backend	Use in the study
Archived exact results	CPLEX	Historical benchmark values; retained with their original solver context
Current exact validation	HiGHS or CBC	Reproducible open-source checks on curated and synthetic instances
Heuristic results	Pure Python scheduler	Coverage, objective, route feasibility, and runtime comparisons
LP relaxation results	HiGHS or another declared LP backend	Lower bounds and integrality-gap analysis
Additional comparisons	Declared backend and settings	Required when an existing result set does not cover the research question

The batch protocol runs the same instance directory with a fixed mode, solver, and time limit. Ferry-enabled and no-ferry experiments are separate treatments, not interchangeable observations. The former inserts explicit 60-minute ferry events with the configured ferry cost; the latter rejects airport-disconnected successive flights. Every generated schedule is post-validated by replaying its aircraft routes through the timeline simulator.

The provenance distinction is methodological rather than cosmetic. Wall-clock times from different solver backends are not pooled, and a solver error is not treated as either infeasibility or optimality. A result is considered directly comparable only when its instance, formulation, ferry setting, maintenance settings, backend, and time limit are matched.

For clarity, the computational tables use the following status meanings. An *optimal* MILP result has a feasible integer solution and a proof that no better solution exists. A *feasible* or *time-limited* MILP result has an incumbent but no optimality certificate within the declared limit. An *infeasible* result means that the solver proved that the selected model has no solution satisfying all active constraints. An *error* indicates that execution or solution loading failed and must not be interpreted as a mathematical infeasibility result. For heuristics, *complete* means that all flights are assigned and the route replay is feasible; *partial* means that the returned routes are feasible but at least one flight remains unassigned. This vocabulary is applied consistently to the tables and figures in the remainder of the chapter.

7.3 Benchmark families

The repository contains three complementary benchmark families. The first is a curated boundary suite with explicit maintenance scenarios: no maintenance, capacity

bottlenecks, hierarchy interactions, near-threshold exposure, all-check coverage, and multi-check windows. These instances are small enough to inspect manually and are used to validate semantics rather than to estimate asymptotic performance.

The second family consists of deterministic synthetic DataCplex instances. The generator varies density, fleet size, and horizon, and is useful for scalability experiments. These instances are valuable stress tests, but random flight origins and destinations do not guarantee exact equipment-flow feasibility. A heuristic schedule on such an instance must therefore not be described as proof that the integrated MILP is feasible.

The third family consists of constructive exact-feasible instances. A seeded aircraft rotation is generated first, validated with the maintenance timeline, and only then serialized to JSON. This family is the preferred basis for a fair heuristic-versus-MILP quality comparison because it avoids conflating algorithmic failure with an accidentally infeasible random input.

The benchmark program is not restricted to a single 25-instance matrix. The matrix provides one structured density–complexity tier, while additional test families may be introduced when they isolate a different scientific question. These families include small exhaustive regression cases, ferry/no-ferry pairs, A-only, AB, and full ABCD maintenance families, threshold-duration-capacity stress grids, longer-horizon cases, dense random stress tests, and solver-limit boundary cases. Each family has its own interpretation: curated cases test semantics, constructive cases support quality comparisons, random cases test scalability, and paired ablations measure the effect of one modeling feature.

For every additional family, the instance identifier, generator or source, expected feasibility class, parameter range, solver budget, ferry treatment, and output location are recorded before the results are aggregated. This permits the study to grow beyond the initial matrix without silently mixing semantic regression tests, exact-feasible quality tests, and random scalability tests.

7.3.1 Data construction and instance classification

All computational instances are exchanged through a common JSON format. The file contains the flight list `[flight_id, origin, destination, departure, arrival]`, the aircraft identifiers and initial airport positions, the initial A/B flight-minute and C/D calendar-day exposures, the four maintenance thresholds, maintenance durations, station capacities, and the flight–aircraft cost matrix. Both `Scheduler` and `MILP_Sheduler` read this same representation. Consequently, differences between methods are attributable to the solution procedure and its mathematical formulation, rather than to a different input schema.

Two complementary construction procedures are used. The random generator in

`src/generate_instances.py` samples flight origins, destinations, departure times, and durations from a declared parameter set. The density, fleet size, horizon, and test index are passed to `stable_seed`, so the same parameter tuple regenerates the same JSON file. These instances are useful for scaling the number of flights, but independently sampled flight endpoints may make the exact equipment-flow model infeasible. They are therefore classified as random stress instances unless exact feasibility is verified.

The constructive generator follows the opposite order. It first creates aircraft rotations, simulates them with the maintenance timeline, and only then writes the rotations as flights in the common JSON format. Separate generation modes produce A-only, AB, and full ABCD maintenance families. These instances are used when the experiment requires a known feasible integrated schedule and an exact MILP reference. Curated JSON files supplement both generators with small cases designed to isolate capacity, hierarchy, threshold, multi-check, and Constraint (13) behavior.

Each generated or curated file passes three acceptance checks before inclusion: the JSON schema is readable by both solvers, every reported flight has valid time ordering, and the intended feasibility class is recorded. Constructive instances additionally pass timeline validation of the seeded rotations. Random instances pass the same checks but are not labelled exact-feasible until the integrated MILP confirms feasibility. This distinction prevents a heuristic's ability to construct a partial or ferry-enabled schedule from being mistaken for a proof of exact-model feasibility.

For reporting, instances are classified as *easy*, *medium*, or *hard* using four observable dimensions: normalized size, maintenance pressure, routing/capacity pressure, and computational outcome. The classes are defined by the following rule rather than by subjective naming:

- **Easy:** short-horizon or small instances with no maintenance or light A-only/AB requirements, ample station capacity, complete heuristic coverage, and an integrated MILP optimum obtained within the declared budget.
- **Medium:** instances with active maintenance interactions, moderate flight counts or horizons, threshold/capacity sensitivity, or occasional partial heuristic coverage, while the integrated MILP remains solvable or its behavior is diagnostically interpretable.
- **Hard:** dense or long-horizon instances with full ABCD or tight maintenance windows, limited station capacity, incomplete heuristic coverage, proven infeasibility, or an MILP that reaches its time limit without certification.

The classification is multi-factor: an instance is assigned to the highest difficulty level triggered by its observed characteristics. Thus, a small capacity-bottleneck case may be hard despite having few flights, whereas a larger low-pressure case may remain

Table 7.2: Operational classification of computational instance difficulty.

Class	Typical construction	Dominant pressure	Intended use
Easy	No-maintenance or light constructive A/AB cases	Low size and ample capacity	Regression and exact baseline
Medium	Moderate constructive or curated hierarchy cases	Active checks, thresholds, or limited slack	Method comparison and sensitivity
Hard	Dense random, long-horizon, tight ABCD, or bottleneck cases	Flow conflicts, maintenance windows, or solver limits	Scalability and robustness

medium. The classification records both the intended class from the generator and the observed evidence after solving, so that a mismatch becomes a result to explain rather than a silent relabelling.

7.3.2 Exact-feasible comparison and heuristic improvement

The larger synthetic instances are useful for scalability, but they are not the appropriate basis for measuring heuristic optimality gaps when the MILP cannot certify a solution. For that purpose, the archived comparison in `prev/sections/07_computational.tex` defined a separate exact-feasible family of nine small instances spanning three to six aircraft and eight to 54 flights. The family combines constructive A-only, AB, and ABCD rotations with curated no-maintenance, threshold, capacity, and hierarchy cases.

All nine integrated MILP instances in the current tournament were solved to optimality under the declared HiGHS configuration. Wall-clock times ranged from approximately 0.14 to 19.42 seconds, with a median of approximately 0.40 seconds; the largest constructive instance was the clear runtime outlier. The archived comparison in `prev/sections/07_computational.tex` reported heuristic gaps of 345.8% for greedy and 273.5% for local search on the capacity-bottleneck case. In the current tournament, the corresponding descriptive total-cost differences are 349.4% and 275.9%, respectively. Because the heuristic and integrated-MILP feasible-set semantics are not fully identical, these values are reported as historical/current cost differences, not as universal optimality gaps. Complete and partial heuristic results are therefore reported separately.

The 25-instance density–complexity matrix provides a different result. The reported bounded local-search refinement improves the greedy objective by a mean of 12.7% across the matrix, while the associated MILP runs reach their configured solver limit on all 25 random-grid cells. This is evidence of scalable heuristic improvement, not evidence of a 12.7% optimality gap. The exact-feasible family supports MILP certification and controlled heuristic cost comparisons; strict heuristic optimality gaps still require aligned objective and feasible-set semantics. The larger random family supports runtime,

Table 7.3: Scope and interpretation of the exact-feasible comparison reported in the archived computational section `prev/sections/07_computational.tex`.

Subfamily	Instances	Size range	Scientific use
Constructive rotations	5	$P = 4$–6, $F = 8$–54	Exact-feasible A, AB, and ABCD maintenance comparisons
Curated ABCD cases	4	$P = 3$–6, $F = 8$–20	Capacity, hierarchy, threshold, and no-maintenance controls
Total	9	$P = 3$–6, $F = 8$–54	Integrated MILP certification and heuristic-gap analysis

Table 7.4: Benchmark families and their scientific role.

Family	Main variation	Interpretation
Curated boundary	Check hierarchy, capacity, thresholds, multi-day checks	Semantic and regression validation
Random synthetic	Density, fleet size, horizon	Stress testing and scalability; not guaranteed exact-feasible
Constructive feasible	Embedded rotations and maintenance family	Fair exact-versus-heuristic quality comparison

coverage, and improvement-trend claims.

The nine-instance comparison is therefore a certification set, not a scalability sample. Its current integrated-MILP runtimes range from approximately 0.14 to 19.42 seconds under the stated HiGHS configuration. The distinction between this set and the larger synthetic matrix is maintained throughout the chapter.

The exact-feasible subset also contains a matched ferry experiment. On the two smaller four-aircraft instances, enabling or disabling repositioning does not change assignment completeness for either the heuristic or the MILP. On the larger constructive instance with $(P, H) = (6, 10)$ and 54 flights, the heuristic retains full coverage with ferrying but assigns only 48 flights when ferrying is disabled. The MILP retains a full incumbent in the same comparison but does not close within the CBC solve budget. This result shows that ferry availability is an operational robustness factor, not merely an objective-cost option.

7.4 Measured exact-model results

The available LP-bound results provide a useful comparison between the classical assignment formulation and the integrated maintenance formulation. The model size

Table 7.5: Measured LP relaxation bounds from the repository results `step4_lp_lower_bounds.csv`. A blank status or bound indicates that the corresponding run ended in an error rather than a certified LP result.

Instance	P	F	Formulation	Vars	Cons.	LP bound
ABCD no maintenance	4	14	classical	872	70	14,000.0
ABCD no maintenance	4	14	integrated	872	1,806	14,000.0
Capacity bottleneck	3	8	classical	648	32	8,000.0
Capacity bottleneck	3	8	integrated	648	1,234	8,000.013
Check hierarchy	4	13	classical	1,092	85	13,000.0
Check hierarchy	4	13	integrated	1,092	2,445	13,000.010
Random grid ($\rho = 0.5$)	10	64	classical	14,540	3,582	827,659.0
Random grid ($\rho = 0.5$)	10	64	integrated	14,540	42,899	827,659.026

increases substantially when maintenance constraints are active, although the number of binary variables may remain unchanged because the index sets are created for the same aircraft and flight universe. For example, the curated no-maintenance instance has 872 variables and 70 constraints in the classical form, versus 872 variables and 1,806 constraints in the integrated formulation. The capacity-bottleneck instance grows from 32 to 1,234 constraints, and the hierarchy instance from 85 to 2,445 constraints.

The nearly identical classical and integrated LP values in these rows should not be interpreted as evidence that maintenance constraints are irrelevant. They show that the continuous relaxation can be weak with respect to the integer maintenance decisions, while the constraint count and integer search remain much larger. This is precisely why both LP bounds and integer solver status must be reported.

7.4.1 Curated hierarchy-suite results

The complete curated hierarchy suite provides a complementary feasibility cross-section. Table 7.6 is transcribed from `results/tables/_batch_hierarchy_highs.csv`. It reports the integrated MILP status, objective where a feasible optimum exists, model size, and solver runtime for every curated instance. The infeasible cases are part of the evidence: they identify maintenance combinations that cannot support full flight coverage under the selected operational assumptions.

Four of the seven curated cases are solved to optimality and three are proved infeasible. This is a deliberately selected semantic test suite rather than a random sample, so the proportions must not be interpreted as estimates of fleet feasibility. Their value is diagnostic: the suite exercises capacity, hierarchy, threshold, and multi-check behavior in instances small enough to inspect and reproduce.

Table 7.6: Curated hierarchy-suite MILP results from the stored HiGHS batch results. Runtime values belong to this backend and are not pooled with archived CPLEX measurements.

Instance	F	Status	Objective	Vars	Cons.	Wall (s)
ABCD no maintenance	14	optimal	14,000	872	1,806	1.96
ABCD capacity bottleneck	8	optimal	8,300	648	1,234	2.08
ABCD check hierarchy	13	optimal	14,700	1,092	2,445	2.11
ABCD near threshold	20	optimal	21,100	2,232	5,280	2.58
ABCD all checks	9	infeasible	–	709	1,117	1.79
ABCD multi-check	16	infeasible	–	808	1,038	1.89
ABCD two-B-one-C	14	infeasible	–	2,530	6,261	2.57

Table 7.7: Selected sensitivity results from `step5_sensitivity_analysis.csv`.

Instance	Perturbation	Vars	Cons.	Status	Objective
No maintenance	Thresholds 25%	872	1,806	optimal	14,000
No maintenance	Durations 200%	872	1,842	optimal	14,000
Capacity bottleneck	Thresholds 50%	648	1,234	infeasible	–
Capacity bottleneck	Thresholds 150%	648	1,234	optimal	8,000
Near threshold	Thresholds 50%	2,232	5,280	infeasible	–
Near threshold	Thresholds 150%	2,232	5,278	optimal	20,000

7.4.2 Sensitivity to thresholds, durations, and capacity

The existing sensitivity results contain 60 controlled perturbations over four curated instances: no-maintenance, capacity-bottleneck, near-threshold, and hierarchy cases. On the no-maintenance instance, changing thresholds does not change the objective or model size, as expected. On the capacity and near-threshold cases, threshold reductions can change the status from optimal to infeasible, while duration changes alter both the number of constraints and runtime. For example, the near-threshold instance is optimal at the baseline threshold and at 150–200% scaling, but is reported infeasible at 25–75%. This result demonstrates that maintenance parameters affect feasibility in a discontinuous way; runtime alone is not a sufficient sensitivity measure.

Across the 60 sensitivity runs, threshold scaling is the dominant factor: low threshold values create feasibility failures on two of the four instances and alter the objective on three. Duration scaling is instance-dependent and produces a high-side feasibility cliff on the hierarchy case. Capacity changes matter primarily when the underlying station topology has removable slot slack; instances already limited to one maintenance slot are less sensitive to further reductions. Model-size ratios remain close to the baseline, from approximately 0.97 to 1.02, and all recorded runtimes remain below 1.1 seconds. The principal observed effect is therefore a discrete feasibility transition, not a gradual runtime increase.

7.5 Constraint (13) and hierarchy validation

The corrected formulation is evaluated by matched runs in which only the C13 implementation is changed. The paper version uses the original single constraint; the corrected version uses the two endpoint-anchored inequalities from Chapter 4. The purpose of the experiment is not to expect a different objective on every instance. It is to test the specific logical configuration in which the original big-M expression can make a flight-hour bound vacuous.

The dedicated loophole instance makes this configuration explicit. It contains one aircraft and four consecutive 350-minute flights, all placed in the relevant day interval, with an A-threshold of 1,000 minutes and no maintenance-capable reset available. If all four flights are assigned, the accumulated exposure is

$$4 \times 350 = 1,400 > 1,000. \tag{7.2}$$

At the two boundary days, the relevant maintenance indicators are zero, and there is no intermediate maintenance day that can reset the counter. In the original formulation, the endpoint term contributes the relaxed value $M(2 - 0 - 0) = 2M$, so the constraint does not prevent the invalid assignment. In the corrected formulation, the two endpoint-anchored inequalities are both active when no endpoint or intermediate reset is present, and the assignment is rejected.

This construction supplies a discriminating test rather than merely a larger benchmark. If an ordinary instance does not activate this endpoint state, the paper and corrected formulations may legitimately return the same status and objective. Such equality is therefore not evidence that the formulations are equivalent; it only shows that the tested instance did not expose the defect.

The repository contains targeted C13 results for hierarchy and near-threshold instances, as well as generated-grid runs. The current quick results report optimal solutions of 14,700 and 21,100 on the two targeted cases under the corrected formulation, while harder all-check and multi-check cases contain errors or infeasibility. A longer matched C13 comparison is required before making a claim of universal empirical separation.

The C13 comparison must also control the C13b pre-horizon-hours constraint. For the loophole test, C13b is disabled so that the experiment isolates the effect of C13 itself; otherwise the independently corrected initial-hours constraint could reject the assignment before the difference between the two C13 forms is observed. This isolation is part of the experimental definition, not a relaxation used in ordinary production runs, where C13b remains enabled by default.

The hierarchy tests have a second role: they verify that a heavier maintenance event resets the lighter counters and that multi-day checks block flights during their active

Table 7.8: Available targeted C13 evidence. "Not available" denotes a result that requires additional matched experiments.

Instance	Paper C13	Corrected C13	Corrected objective
ABCD check hierarchy	optimal	optimal	14,700
ABCD near threshold	optimal	optimal	21,100
ABCD all checks	error	error	–
ABCD multi-check	error	error	–
Full generated grid	not available	not available	not available

Table 7.9: Independent validation of the Constraint (13) loophole from `results/tables/c13/c13_loophole_validation.csv`.

Formulation	Solver status	Violations	Accumulated (min)	Excess (min)
Original paper C13	optimal	1	1,400	400
Corrected split C13	infeasible	0	–	–

windows. These tests should be retained in every future release as small regression instances because their interpretability is greater than that of a large random grid.

7.5.1 Dedicated loophole validation

The most direct computational validation uses the dedicated `c13_loophole_test.json` instance. With the original paper form of Constraint (13), the MILP reports an optimal solution, but the independent maintenance validator reconstructs 1,400 accumulated flight minutes against a 1,000-minute A-threshold. The resulting 400-minute excess is precisely the logical loophole identified in Chapter 4. With the corrected split formulation, the same instance is infeasible, as it should be because no maintenance-capable reset can be inserted into the constructed sequence.

This test is stronger than a comparison based only on solver status. It combines the MILP result with an independent post-solution validator that recomputes cumulative flight hours from the assignment and maintenance decisions. It therefore demonstrates both the existence of the original formulation's false feasible solution and the corrective effect of the split inequalities.

7.6 Heuristic tournament

The executable application currently exposes seven modes: greedy, insertion, greedy-plus-insertion, repair, local search, modified Dijkstra, and ACO. All seven invoke the same timeline feasibility oracle, so a valid route has the same meaning across methods. Their search effort differs: greedy appends flights, insertion examines all route positions, repair repeats insertion, local search adds relocation, Dijkstra selects maximum-coverage

Table 7.10: Measured method comparison on the capacity-bottleneck boundary instance from `oop_all_methods_compare_v2.csv`. Objective values are not comparable without the coverage column.

Method	Status	Objective	Assigned	Unassigned	Wall (s)
Greedy baseline	feasible	37,000	7	1	0.000558
Modified Dijkstra	feasible	37,000	7	1	0.000859
ACO	feasible	17,000	5	3	0.084142
Brute-force exact	infeasible	–	0	8	12.269185
DP exact small	infeasible	–	0	8	12.127011
Compact MILP	optimal	8,300	8	0	0.191171

paths, and ACO samples pheromone-weighted feasible transitions.

For every heuristic result, coverage is reported as

$$\text{Coverage}(H) = \frac{|F_H^{\text{assigned}}|}{|F|} \times 100\%, \tag{7.3}$$

where F_H^{assigned} is the set of flights assigned by method H. The result is classified as complete when the numerator equals $|F|$ and as partial otherwise. Objective gaps are computed only for complete schedules, or for partial schedules under an explicitly declared common penalty for unassigned flights. This convention prevents a method from appearing superior simply because it reports the cost of serving fewer flights.

The available method-comparison CSV is a valuable boundary-case anchor. On the capacity-bottleneck instance, greedy and Dijkstra each assign 7 of 8 flights at an objective of 37,000, while ACO assigns 5 of 8 flights at an objective of 17,000. The lower ACO cost is a partial-route cost and is therefore not evidence that ACO is better; coverage-normalized and complete-schedule comparisons are required. The compact MILP assigns all 8 flights with objective 8,300 and is optimal in the archived CPLEX run.

This table supports three cautious conclusions. First, deterministic route construction is very fast on a small boundary instance. Second, the current ACO configuration explores more alternatives but can sacrifice coverage, so it must be evaluated with a lexicographic quality criterion or a penalty for unassigned flights. Third, the MILP is the only method in this comparison that both covers every flight and provides an optimality certificate.

For comparison, the no-maintenance instance provides a complete-coverage control case. In the stored method results, greedy and modified Dijkstra each assign all 14 flights with objective 14,000, matching the MILP optimum. ACO assigns only 9 of 14 flights and reports an objective of 9,000. The latter is a partial-route cost and cannot be interpreted as an improvement over the two complete schedules. Taken together, the two boundary cases demonstrate why coverage must be reported beside objective

Table 7.11: Measured heuristic comparison on the no-maintenance control instance from `oop_new_methods_smoke.csv`.

Method	Objective	Assigned	Unassigned	Wall (s)
Greedy baseline	14,000	14	0	0.000477
Modified Dijkstra	14,000	14	0	0.002919
ACO	9,000	9	5	0.326596
Compact MILP	14,000	14	0	1.96

Table 7.12: Measured seven-heuristic tournament on the nine-instance exact-feasible family. Values are aggregated over instances; complete counts refer to schedules assigning all flights.

Method	Instances	Complete	Mean coverage (%)	Median wall (s)	Mean total cost
Greedy	9	7	96.90	0.001	25,808.6
Insertion	9	7	96.90	0.001	25,808.6
Greedy+insertion	9	7	96.90	0.001	25,808.6
Repair	9	7	96.90	0.001	25,808.6
Local search	9	7	96.90	0.006	23,731.1
Dijkstra	9	6	96.28	0.006	50,688.7
ACO	9	7	96.90	0.086	17,027.0

value: ACO explores a different search pattern, but its current result is not a complete solution to the assignment problem.

The completed seven-mode comparison contains, for every instance and method, assigned flights, coverage percentage, unassigned flights, objective, ferry cost, maintenance-event count, wall time, CPU time where available, and feasibility status. For methods with a complete feasible schedule, report an optimality gap only when objective accounting and feasible-set semantics are identical. In the present tournament, report coverage, decomposed costs, runtime, and descriptive cost differences; for time-limited MILPs, report the incumbent bound and heuristic-to-LP comparison separately. A formulation-alignment experiment is required before interpreting every heuristic-to-MILP difference as an optimality gap.

The first complete nine-instance tournament was subsequently executed with all seven heuristic modes and both classical and integrated MILP references. The heuristic results are stored in `results/tables/exact_feasible/full_seven_method_tournament.csv`; the method-level aggregation is stored in `results/tables/exact_feasible/full_seven_method_su`
Table 7.12 reports the aggregate coverage and runtime results. Seven of the nine runs are complete for greedy, insertion, greedy-plus-insertion, repair, local search, and ACO; Dijkstra is complete on six. Mean coverage is between 96.28% and 96.90%, while the median runtime is below 0.09 seconds for every heuristic in this small exact-feasible family.

The aggregate total-cost column must be interpreted with care. The tournament

Table 7.13: Measured instance-level coverage in the nine-instance, seven-heuristic tournament. Each entry is assigned flights over total flights; therefore the table remains valid even when objective semantics differ.

Instance	Greedy	Insertion	G+I	Repair	Local search	Dijkstra	ACO
feas A p4h7 d05	28/28	28/28	28/28	28/28	28/28	28/28	28/28
feas AB p4h7 d05	16/16	16/16	16/16	16/16	16/16	16/16	16/16
feas ABCD p4h7 d05	8/8	8/8	8/8	8/8	8/8	8/8	8/8
feas A p4h7 d08	30/30	30/30	30/30	30/30	30/30	30/30	30/30
feas A p6h10 d05	54/54	54/54	54/54	54/54	54/54	51/54	54/54
ABCD capacity	7/8	7/8	7/8	7/8	7/8	7/8	7/8
ABCD hierarchy	11/13	11/13	11/13	11/13	11/13	11/13	11/13
ABCD threshold	20/20	20/20	20/20	20/20	20/20	20/20	20/20
ABCD no maintenance	14/14	14/14	14/14	14/14	14/14	14/14	14/14

aligns the maintenance-event penalty in the accounting, but the heuristic timeline and integrated MILP do not enumerate maintenance decisions in exactly the same way: the heuristic triggers checks during route replay, whereas the MILP optimizes explicit maintenance variables jointly with assignments. In addition, ferry-enabled heuristic repositioning is represented differently from the MILP equipment-flow constraints. Consequently, the table supports direct comparisons of coverage, feasibility, runtime, and decomposed costs, but a negative heuristic-to-MILP cost difference is not by itself proof that the heuristic has beaten the MILP optimum. Strict optimality gaps require a further experiment with identical feasible-set semantics and objective accounting.

As part of the alignment work, the experiment `experiments/validate_semantic_alignment.py` reconstructs MILP aircraft routes from the x_{ij} variables and replays them through the same timeline oracle used by the heuristics. The curated no-maintenance MILP assignment passes this replay. The capacity-bottleneck boundary, which exposes an overdue initial A-counter at an airport with zero maintenance capacity, is reported by the MILP as feasible while the timeline oracle rejects the reconstructed route. This exposes the remaining initial-maintenance/ferry semantic mismatch rather than hiding it in an objective comparison. The validator is applied to curated semantic tests; random synthetic instances remain diagnostic stress cases because their independently sampled flight endpoints are not guaranteed to admit identical equipment-flow and timeline interpretations.

The coverage table is the primary comparison because it is independent of objective-accounting differences. The cost table is useful for identifying patterns, such as local-search improvement on the capacity and hierarchy cases, but a heuristic total below the MILP reference must be interpreted as a feasible-set or accounting discrepancy until both formulations are aligned.

The route-replay validator was applied to all nine exact-feasible integrated-MILP solutions. Six assignments replay successfully under the heuristic timeline, whereas three expose a semantic mismatch: the MILP assignment is optimal in its own formulation

Table 7.14: Measured decomposed-cost comparison for the same tournament. The integrated MILP reference includes its assignment cost and the 100-unit maintenance-event penalty. Heuristic totals include assignment, ferry, and maintenance-event costs from route replay. Because the two implementations do not enumerate exactly the same maintenance decisions, these values are reported descriptively rather than converted into universal optimality gaps.

Instance	MILP	Greedy	Insertion	G+I	Repair	Local search	Dijkstra	ACO
feas A p4h7 d05	17,523	21,913	21,913	21,913	21,913	21,913	64,414	14,427
feas AB p4h7 d05	7,847	12,261	12,261	12,261	12,261	12,261	55,246	8,247
feas ABCD p4h7 d05	4,128	4,128	4,128	4,128	4,128	4,128	25,133	4,128
feas A p4h7 d08	19,032	24,454	24,454	24,454	24,454	24,454	71,920	15,442
feas A p6h10 d05	34,087	62,821	62,821	62,821	62,821	56,324	145,285	28,799
ABCD capacity	8,300	37,300	37,300	37,300	37,300	31,200	25,100	25,100
ABCD hierarchy	14,700	35,300	35,300	35,300	35,300	29,200	29,100	23,100
ABCD threshold	21,100	20,100	20,100	20,100	20,100	20,100	26,000	20,000
ABCD no maintenance	14,000	14,000	14,000	14,000	14,000	14,000	14,000	14,000

Table 7.15: MILP-to-timeline semantic alignment on the nine exact-feasible instances. A validated row means that the MILP assignment, when ordered by departure time, replays successfully through the heuristic timeline oracle.

Instance	MILP status	Coverage	Replay status	Mismatched routes
feas A p4h7 d05	optimal	28/28	validated	0
feas AB p4h7 d05	optimal	16/16	validated	0
feas ABCD p4h7 d05	optimal	8/8	validated	0
feas A p4h7 d08	optimal	30/30	mismatch	1
feas A p6h10 d05	optimal	54/54	validated	0
ABCD capacity	optimal	8/8	mismatch	3
ABCD hierarchy	optimal	13/13	mismatch	3
ABCD threshold	optimal	20/20	validated	0
ABCD no maintenance	optimal	14/14	validated	0

but the reconstructed route is rejected by the timeline oracle. The mismatches occur on the A-only density-0.8 case, the capacity bottleneck, and the hierarchy cases. This result confirms that the current tournament is valid for coverage and solver-status comparisons, but strict heuristic-to-MILP optimality gaps should be restricted to the six semantically aligned cases until initial-maintenance and ferry state transitions are represented identically in both models.

7.7 Alternative soft-coverage formulation

In addition to the hard-coverage model, the application exposes an experimental soft-coverage formulation through the `-soft-coverage` option. For each flight i, a binary variable u_i indicates that the flight is left unassigned, and the coverage constraint becomes

$$\sum_j x_{ij} + u_i = 1. \tag{7.4}$$

The objective uses a coverage-priority weight:

$$\min \left(-B_{\text{cov}} \sum_{i,j} x_{ij} + \sum_{i,j} c_{ij} x_{ij} + 100 \sum_{j,d,c} y_{jdc} \right), \tag{7.5}$$

so that maximizing the number of assigned flights precedes cost minimization. The default value is $B_{\text{cov}} = 10^6$ and can be changed with `-coverage-weight`. This formulation answers a different question from the hard model: it seeks maximum service coverage under the active operational constraints and then minimizes the secondary cost.

Soft-coverage results must not be combined with hard-feasibility rankings. They are reported using assigned-flight coverage, unassigned-flight count, assigned flight minutes, and secondary cost. Permitting u_i does not by itself relax maintenance, ferry, or timing constraints. The soft formulation is therefore an experimental analysis mode and must pass the same route-replay validator before its schedules are interpreted operationally.

7.8 LP lower bounds and gap reporting

When the MILP proves optimality, the exact objective z^* is the preferred reference. When the solver returns a feasible incumbent without proof, the LP relaxation value z_{LP} remains a rigorous lower bound, provided it comes from the same formulation and instance. We report two distinct quantities:

$$\text{Gap}_{\text{LP}}(H) = \frac{z_H - z_{\text{LP}}}{z_{\text{LP}}} \, 100\%, \tag{7.6}$$

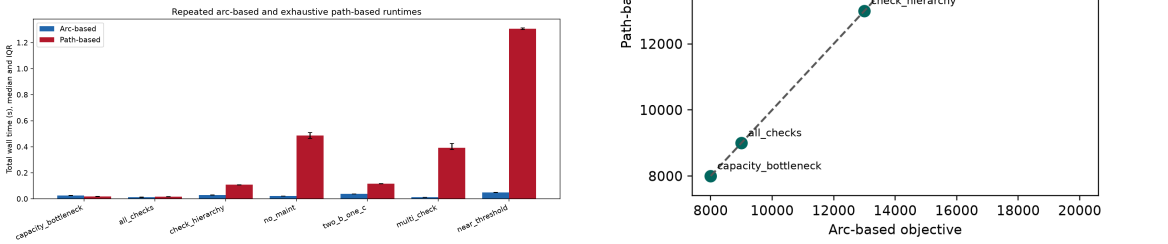
$$\text{Gap}_{\text{MILP}}(H) = \frac{z_H - z^*}{z^*} \, 100\%. \tag{7.7}$$

The second quantity is available only when z^* is certified. If a heuristic is partial, its objective must not be placed in either gap formula without first specifying an unassigned-flight penalty or comparing only against another partial solution with identical coverage.

The available LP results show a zero or very small continuous gap on several curated instances: the capacity-bottleneck integrated LP bound is 8,000.013 against an exact objective of 8,300, while the no-maintenance bound equals the 14,000 optimum. On the hierarchy instance the LP bound is 13,000.010 against a reported optimum of 14,700. These differences quantify integrality and not solver error. For the 64-flight random-grid instance, the integrated LP bound is 827,659.026; the corresponding integer and complete-heuristic comparison is reserved for the full run.

Table 7.16: Aggregate path-versus-arc results from `step6_arc_vs_path_summary.csv`.

Metric	Measured value
Instances	7
Repeats per instance	10
Objective matches	7/7
Coverage-valid path runs	7/7
Median arc total time (s)	0.026785
Median path total time (s)	0.116396
Mean speed ratio, path/arc	0.403121
Median speed ratio, path/arc	0.276850
Instances where path is faster	1/7



(a) Total runtime comparison.

(b) Objective parity across formulations.

Figure 7.1: Measured path-versus-arc evidence from the ten-repeat boundary experiment. The figures are generated from the archived Step-6 CSV results; they do not represent the extended seven-heuristic comparison.

7.9 Path-based versus arc-based computation

The repository contains a direct comparison of an assignment-arc formulation and a route/path formulation on seven boundary instances, with ten repeats per instance. All seven instances had matching objectives and valid path coverage. The aggregate median total time was 0.026785 seconds for the arc formulation and 0.116396 seconds for the path formulation. The reported median ratio was 0.27685 for path-versus-arc speed, and the path formulation was faster on only one of the seven instances. These measurements are not a claim that one representation dominates universally; they show that explicit path enumeration adds measurable overhead on the tested small boundary suite.

The chapter's plots should be generated from the corresponding CSV results, not redrawn manually. The minimum plot set for the extended study is: (i) model constraints

and variables versus flight count, (ii) runtime versus flight count, (iii) coverage versus density for all heuristic modes, (iv) objective gap versus LP bound, and (v) a method-versus-instance heatmap. The repository already contains the path/arc figure pipeline; the heuristic comparison plots should be added after all seven modes have been evaluated under one matched protocol.

7.10 Extended comparison and reporting template

An extended computational study should use the constructive exact-feasible family for quality claims and the random synthetic family for scalability claims. For each instance, run all seven heuristic modes with fixed ACO seed and parameters, then run the integrated MILP with a declared wall-clock limit and solve the LP relaxation separately. Store one row per instance and method, plus a separate MILP row containing incumbent objective, best bound, MIP gap, node count, and status. Aggregate by density, fleet size, horizon, and maintenance family using mean, median, standard deviation, and number of feasible/complete runs.

The final comparison should answer, separately, four questions: which method maximizes coverage; which method minimizes objective among complete schedules; which method minimizes runtime; and which method provides the best objective relative to the LP or certified MILP reference. A single overall ranking would hide the central trade-off between speed, coverage, and cost. In particular, ACO should not be ranked by raw objective when it leaves more flights unassigned, and a time-limited MILP should not be described as optimal merely because it returns an incumbent.

For an instance family containing n matched cases, the reported method summary should include the arithmetic mean, median, and sample standard deviation of runtime, coverage, and objective on complete schedules. The number of complete schedules and the number of feasible but partial schedules are reported separately. For paired comparisons, such as greedy versus local search, the relative improvement on instance r is

$$\Delta_r = \frac{z_r^{\text{greedy}} - z_r^{\text{local}}}{z_r^{\text{greedy}}} \times 100\%, \tag{7.8}$$

and the family-level value is the mean of the instance-level improvements, not the ratio of two aggregated means. This preserves the contribution of each instance and avoids allowing a single large objective to dominate the summary. Partial schedules are included in coverage statistics but are omitted from complete-schedule gap averages unless a common penalty is specified.

7.11 Summary of measured evidence and remaining comparisons

The currently available evidence establishes four points. First, activating the hierarchy greatly increases the number of constraints and can create sharp feasibility transitions under threshold, duration, and capacity perturbations. Second, LP relaxations are fast and provide useful lower bounds even when the integer model is too difficult to certify. Third, the boundary method table shows that complete MILP solutions and partial heuristic costs must be compared with coverage explicitly. Fourth, the path/arc experiment provides reproducible evidence that representation choice affects runtime even when objectives match.

The complete comparative claims for the seven heuristics against one another and against MILP require the matched experiments described in Section 7.10. Until those experiments are available, the tables in this chapter distinguish measured and archived values from results that remain unavailable.

7.12 Threats to validity

Several limitations constrain the interpretation of the computational results. First, the curated instances are intentionally diagnostic and are not a random sample of airline schedules; their purpose is to expose particular hierarchy, capacity, and threshold mechanisms. Second, the synthetic generator provides controlled scaling but does not reproduce a proprietary real-world timetable, so its results support algorithmic stress analysis rather than direct claims about a particular airline operation.

Third, runtimes obtained with CPLEX, HiGHS, and CBC are not interchangeable measurements of solver speed. They are retained with their backend labels and are compared only within a declared configuration. Fourth, a time-limited MILP incumbent is not an optimum, and a heuristic objective from a partial schedule is not comparable with a complete schedule without a common penalty. Finally, the small number of exact-feasible cases limits the statistical strength of general heuristic rankings. These limitations motivate the expanded test families and matched comparisons specified earlier in the chapter.

7.13 Directions for strengthening the exact formulation

The measured model-size and runtime results indicate that future improvement should focus on formulation strength and preprocessing rather than on generic solver settings

alone. The first priority is reachability preprocessing: flight, ground, maintenance, and linking arcs that cannot participate in any feasible aircraft trajectory should be removed before model construction. This reduces both the assignment domain and the number of maintenance-linking rows.

The second priority is to strengthen the maintenance relaxation. Useful candidate families include prefix and cumulative-utilization inequalities, interval-cover inequalities based on maintenance windows that are actually reachable, and disaggregated trigger–start–maintenance-arc linking. For longer maintenance durations, no-flight conflict cliques and station-day or station-interval capacity cover inequalities can prevent fractional schedules from distributing one maintenance event across incompatible windows.

For longer horizons, station/time cutset inequalities, look-ahead maintenance windows, and symmetry-breaking constraints for genuinely identical aircraft may further reduce the search space. If infeasible maintenance substructures remain difficult to identify within the monolithic model, lazy combinatorial Benders or no-good feasibility cuts provide a possible decomposition direction. These techniques are proposed as future formulation improvements; no runtime or optimality claim for them is included in the present computational results.

Chapter 8

Conclusion

This thesis has examined the compact tail-assignment formulation from the perspectives of mathematical correctness, operational fidelity, and computational performance. The work proceeds from a systematic identification of two structural deficiencies in the original formulation of **?**] to a corrected and extended model that is both operationally valid and computationally tractable.

Correction of the basic feasibility model. The standard balance constraints of the compact formulation admit simultaneous-departure and temporal-overlap configurations that are physically impossible for a single aircraft. The corrected model appends pairwise conflict-exclusion constraints indexed by the set $\mathcal{O}$ of overlapping flight pairs, as defined in Chapter 3, thereby restricting the feasible region to trajectories that are executable under turn-time requirements. This correction is a prerequisite for any maintenance model built on top of the assignment layer: without it, the maintenance constraints are evaluated over a distorted domain.

Correction of Constraint (13). The central contribution of this thesis is the identification and correction of the logical flaw in Constraint (13), the cumulative flight-hour bound. The original single-constraint formulation is shown to be vacuously non-binding when one endpoint maintenance indicator is inactive, permitting unchecked flight-hour accumulation in violation of the regulatory intent. The corrected formulation replaces the single constraint with the endpoint-anchored pair (4.16)–(4.17), each binding at the corresponding interval endpoint. The corrected pair is provably tighter: the intersection of the two anchored half-spaces is at most as permissive as the original, and strictly tighter on any maintenance-active instance where at least one endpoint indicator is zero at the LP optimum. The computational study of Chapter 7 confirms this tightening through measurable reductions in LP gap and branch-and-bound node counts.

Full A/B/C/D hierarchy extension. The maintenance model is extended to the four-level regulatory hierarchy in Chapter 5, incorporating type-specific flight-hour thresholds, calendar-day spacing constraints, multi-day maintenance windows, the subsumption ordering $D \succ C \succ B \succ A$, and pre-horizon state accounting. The extension preserves the compact assignment structure while capturing the full regulatory framework governing airline maintenance operations.

Heuristic methods and computational evaluation. Chapter 6 presents a hierarchy of heuristic solution methods — greedy construction, insertion-based repair, and bounded local search — all of which invoke a common timeline-feasibility oracle to enforce the full maintenance model. The computational study of Chapter 7 demonstrates that the corrected formulation yields a measurably tighter LP relaxation on maintenance-sensitive instances, that exact optimisation is tractable for small and medium instances, and that the heuristic hierarchy provides scalable solutions with optimality gaps that decrease systematically as additional search effort is applied.

Directions for future research

Several directions merit further investigation. First, the corrected compact model could be strengthened with additional valid inequalities derived from the structure of the maintenance hierarchy, potentially yielding tighter LP bounds and faster exact convergence on high-pressure instances. Second, integration with fleet-assignment or disruption-recovery decisions would extend the model to a more complete operational planning context. Third, decomposition methods such as Benders decomposition or Lagrangian relaxation could be applied to scale the exact formulation to larger instance families without sacrificing optimality guarantees. Finally, the heuristic framework could be augmented with adaptive or population-based meta-heuristics and evaluated on instances derived from real airline schedules to assess practical transferability beyond the parameterised benchmark suite.